

ΕΡΓΑΣΙΑ 1

Γραμμική & Συνδυαστική Βελτιστοποίηση

Διορθωμένη και συμπληρωμένη έκδοση

Εμμανουήλ Βήχος

AM: 1100830

Τμήμα Ηλεκτρολόγων Μηχανικών & Τεχνολογίας Υπολογιστών

Περιεχόμενα

Σημείωμα για την παρούσα έκδοση	2
1 Γραφική επίλυση προβλήματος γραμμικού προγραμματισμού	3
1.1 Γιατί επιτρέπεται η αποσύνθεση	3
1.2 Υποπρόβλημα 1: μεταβλητές x_1, x_2	3
1.3 Υποπρόβλημα 2: μεταβλητές x_3, x_4	5
1.4 Συνολική λύση, μοναδικότητα και εκφυλισμός	6
2 Μοντελοποίηση και γραφική επίλυση προβλήματος χαρτοφυλακίου	7
2.1 (α) Μοντελοποίηση	7
2.2 (β) Γραφική επίλυση	7
2.3 Πλεονάζων περιορισμός και σκιάδεις τιμές	7
3 Μετατροπή προβλήματος με απόλυτες τιμές σε γραμμικό πρόγραμμα	10
3.1 Μέθοδος A: διάσπαση μεταβλητών	10
3.2 Μέθοδος B: μετασχηματισμός με άνω φράγματα	10
3.3 Λύση και σύνολο βέλτιστων λύσεων	11
4 Μοντελοποίηση προβλήματος δρομολόγησης MMM	12
4.1 (α) Πλήρης κάλυψη της εκτιμώμενης ζήτησης	12
4.2 (β) Χωρητικότητα στο $[95\%, 105\%]$ της ζήτησης	12
4.3 Αριθμητική επαλήθευση των δύο μοντέλων	13
5 Βασικές λύσεις και κορυφές του πολυτόπου	14
5.1 (α) Όλες οι τομές υπερεπιπέδων και οι κορυφές του πολυτόπου	14
5.2 (β) Όλες οι βασικές λύσεις του συστήματος εξισώσεων	15
5.3 (γ) Αντιστοίχιση βασικών λύσεων $\leftrightarrow$ κορυφών	16
6 Αλγόριθμος Simplex και γράφος γειτνίασης	18
6.1 (α) Εκτέλεση του Simplex	18
6.2 (β) Simplex adjacency graph	19
7 Τεχνική Φάσης I του αλγορίθμου δύο φάσεων	21
7.1 Σύστημα (α)	21
7.2 Σύστημα (β)	22
A' Παράρτημα: πηγαίος κώδικας	24
A'.1 ex_1.py — γραφική επίλυση με αποσύνθεση	24
A'.2 ex_2.py — χαρτοφυλάκιο, πλεονάζοντες περιορισμοί, σκιάδεις τιμές	27
A'.3 ex_3.py — δύο μετασχηματισμοί και σύνολο βέλτιστων λύσεων	30
A'.4 ex_4.py — αριθμητική επαλήθευση των μοντέλων MMM	31
A'.5 ex_5.py — απαρίθμηση τομών, βασικών λύσεων και κορυφών	33
A'.6 ex_6.py — Simplex και πλήρης κατασκευή του adjacency graph	35
A'.7 ex_7.py — Φάση I με tableaux και πιστοποιητικό Farkas	38

Σημείωμα για την παρούσα έκδοση

Το κείμενο αυτό είναι αναθεωρημένη έκδοση της Εργασίας 1. Ξαναδουλεύοντας τις ασκήσεις με βάση τις διαλέξεις, εντόπισα αρκετά σημεία στα οποία η αρχική μου υποβολή ήταν ελλιπής ή λανθασμένη — κυρίως στις Ασκήσεις 1, 5 και 6. Πιστεύω πως έλειπαν τα παρακάτω:

Άσκηση	Τι προστέθηκε / διορθώθηκε
1	Τεκμηριώνεται αυστηρά γιατί επιτρέπεται η αποσύνθεση σε δύο ανεξάρτητα υποπροβλήματα. Εντοπίζονται οι δύο εκφυλισμένες κορυφές $(0, 1)$ και $(2, 0)$, που στην αρχική υποβολή δεν είχαν επισημανθεί. Προστίθεται πίνακας ενεργών περιορισμών ανά κορυφή και αυστηρή απόδειξη μοναδικότητας του βελτίστου.
2	Αποδεικνύεται ότι ο περιορισμός $x_D \leq 10$ είναι πλεονάζων , και δίνονται οι σκιώδεις τιμές με οικονομική ερμηνεία.
3	Δίνονται δύο ισοδύναμοι μετασχηματισμοί (διάσπαση μεταβλητών και μέθοδος άνω φραγμάτων) με πλήρη απόδειξη ισοδυναμίας — συμπεριλαμβανομένου του επιχειρήματος γραμμικής εξάρτησης που εξασφαλίζει $x_j^+ x_j^- = 0$. Επισημαίνεται ότι το πρόβλημα έχει άπειρες βέλτιστες λύσεις .
4	Προστίθεται συζήτηση εφικτότητας, τεκμηρίωση της γραμμικοποίησης του $\min(\cdot)$ στο ερώτημα (β) και αριθμητικό στιγμιότυπο επαλήθευσης.
5	Προστίθενται οι πλήρεις πίνακες και των 35 τομών υπερεπιπέδων (α) και των 35 βασικών λύσεων (β), καθώς και ο πίνακας αντιστοίχισης (γ). Στην αρχική υποβολή δίνονταν μόνο τα συγκεντρωτικά πλήθη.
6	Προστίθεται ο Simplex adjacency graph ως σχήμα, μαζί με την πλήρη απαρίθμηση των 8 κορυφών, των 11 βάσεων και των 17 επιτρεπτών pivots. Τεκμηριώνονται τα <i>εκφυλισμένα pivots</i> στην κορυφή $(1, 1, 1)$.
7	Προστίθεται αλγεβρικό πιστοποιητικό μη-εφικτότητας (Farkas) για το σύστημα (β).
Γενικά	Ο κώδικας παρατίθεται ως κείμενο (listings) αντί για στιγμιότυπα οθόνης, με αρίθμηση γραμμών και σχόλια· όλοι οι υπολογισμοί γίνονται σε <i>ακριβή ρητή αριθμητική</i> (fractions) ώστε ο εντοπισμός των εκφυλισμών να είναι απολύτως αξιόπιστος.

1 Γραφική επίλυση προβλήματος γραμμικού προγραμματισμού

Διατύπωση

$$\max Z = 3x_1 + 7x_2 + 4x_3 - 3x_4$$

$$(\Pi_1) \ x_1 + 2x_2 \leq 6$$

$$(\Pi_5) \ 2x_3 - x_4 \leq 4$$

$$(\Pi_2) \ x_3 + x_4 \leq 6$$

$$(\Pi_6) \ x_3 \geq 2$$

$$(\Pi_3) \ 4x_1 + 5x_2 \leq 20$$

$$(\Pi_7) \ 1 \leq x_2 \leq 2$$

$$(\Pi_4) \ -2x_1 + x_2 \leq 1$$

$$(\Pi_8) \ x_1, x_4 \geq 0$$

1.1 Γιατί επιτρέπεται η αποσύνθεση

Το πρόβλημα έχει 4 μεταβλητές, άρα δεν λύνεται άμεσα γραφικά. Παρατηρούμε όμως ότι *κανένας* περιορισμός δεν αναμειγνύει τις μεταβλητές (x_1, x_2) με τις (x_3, x_4) :

$$\underbrace{\Pi_1, \Pi_3, \Pi_4, \Pi_7, \ x_1 \geq 0}_{\text{μόνο } x_1, x_2} \quad \text{και} \quad \underbrace{\Pi_2, \Pi_5, \Pi_6, \ x_4 \geq 0}_{\text{μόνο } x_3, x_4}$$

Επιπλέον η αντικειμενική συνάρτηση είναι *διαχωρίσιμη*: $Z = \underbrace{(3x_1 + 7x_2)}_{Z_1} + \underbrace{(4x_3 - 3x_4)}_{Z_2}$.

Τεκμηρίωση. Έστω $S_1 \subset \mathbb{R}^2$ η εφικτή περιοχή των (x_1, x_2) και $S_2 \subset \mathbb{R}^2$ αυτή των (x_3, x_4) . Τότε η εφικτή περιοχή του αρχικού προβλήματος είναι ακριβώς το *καρτεσιανό γινόμενο* $S = S_1 \times S_2$. Για κάθε $(u, v) \in S_1 \times S_2$ ισχύει $Z(u, v) = Z_1(u) + Z_2(v)$, οπότε

$$\max_{(u,v) \in S_1 \times S_2} [Z_1(u) + Z_2(v)] = \max_{u \in S_1} Z_1(u) + \max_{v \in S_2} Z_2(v),$$

επειδή οι δύο όροι μεγιστοποιούνται ανεξάρτητα. Άρα $Z^* = Z_1^* + Z_2^*$ και $x^* = (u^*, v^*)$. Η αποσύνθεση είναι λοιπόν *ακριβής*, όχι προσέγγιση.

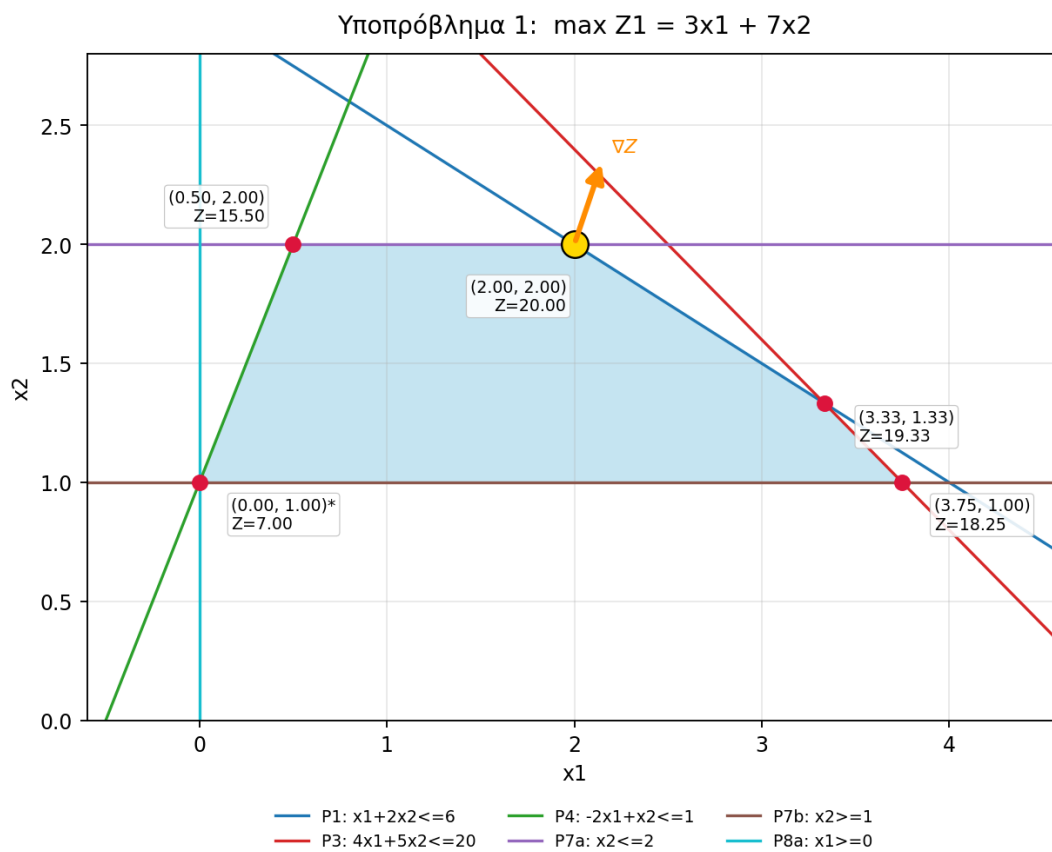
1.2 Υποπρόβλημα 1: μεταβλητές x_1, x_2

$$\max Z_1 = 3x_1 + 7x_2 \quad \text{με} \quad x_1 + 2x_2 \leq 6, \ 4x_1 + 5x_2 \leq 20, \ -2x_1 + x_2 \leq 1, \ 1 \leq x_2 \leq 2, \ x_1 \geq 0.$$

Πίνακας 1: Κορυφές του υποπροβλήματος 1 και οι ενεργοί περιορισμοί τους.

#	x_1	x_2	Ενεργοί περιορισμοί	πλήθος	Z_1
1	0	1	$\Pi_4, \ x_2 = 1, \ x_1 = 0$	3	7
2	$\frac{1}{2}$	2	$\Pi_4, \ x_2 = 2$	2	$\frac{31}{2} = 15.5$
3	2	2	$\Pi_1, \ x_2 = 2$	2	20
4	$\frac{10}{3}$	$\frac{4}{3}$	$\Pi_1, \ \Pi_3$	2	$\frac{58}{3} \approx 19.33$
5	$\frac{15}{4}$	1	$\Pi_3, \ x_2 = 1$	2	$\frac{73}{4} = 18.25$

Εκφυλισμός (δεν είχε επισημανθεί στην αρχική υποβολή). Στον $\mathbb{R}^2$ μια μη εκφυλισμένη κορυφή ορίζεται από την τομή *ακριβώς* 2 ευθειών. Στην κορυφή 1 = (0, 1) τέμνονται **τρεις**: $-2x_1 + x_2 = 1$, $x_2 = 1$ και $x_1 = 0$. Άρα η (0, 1) είναι **εκφυλισμένη κορυφή**. Αν ο Simplex έφτανε εκεί, θα υπήρχαν $\binom{3}{2} = 3$ διαφορετικές βάσεις που αντιστοιχούν στο ίδιο σημείο, και θα ήταν δυνατά εκφυλισμένα pivots (αλλαγή βάσης χωρίς μεταβολή του Z).



Σχήμα 1: Υποπρόβλημα 1. Με γαλάζιο η εφικτή περιοχή, με κόκκινο οι κορυφές, με κίτρινο η βέλτιστη κορυφή και με πορτοκαλί το $\nabla Z_1 = (3, 7)$. Ο αστερίσκος σημειώνει την εκφυλισμένη κορυφή.

Λεπτομέρειες υπολογισμού κορυφών:

$$K_2 : -2x_1 + 2 = 1 \Rightarrow x_1 = \frac{1}{2}; \quad K_3 : x_1 + 2(2) = 6 \Rightarrow x_1 = 2;$$

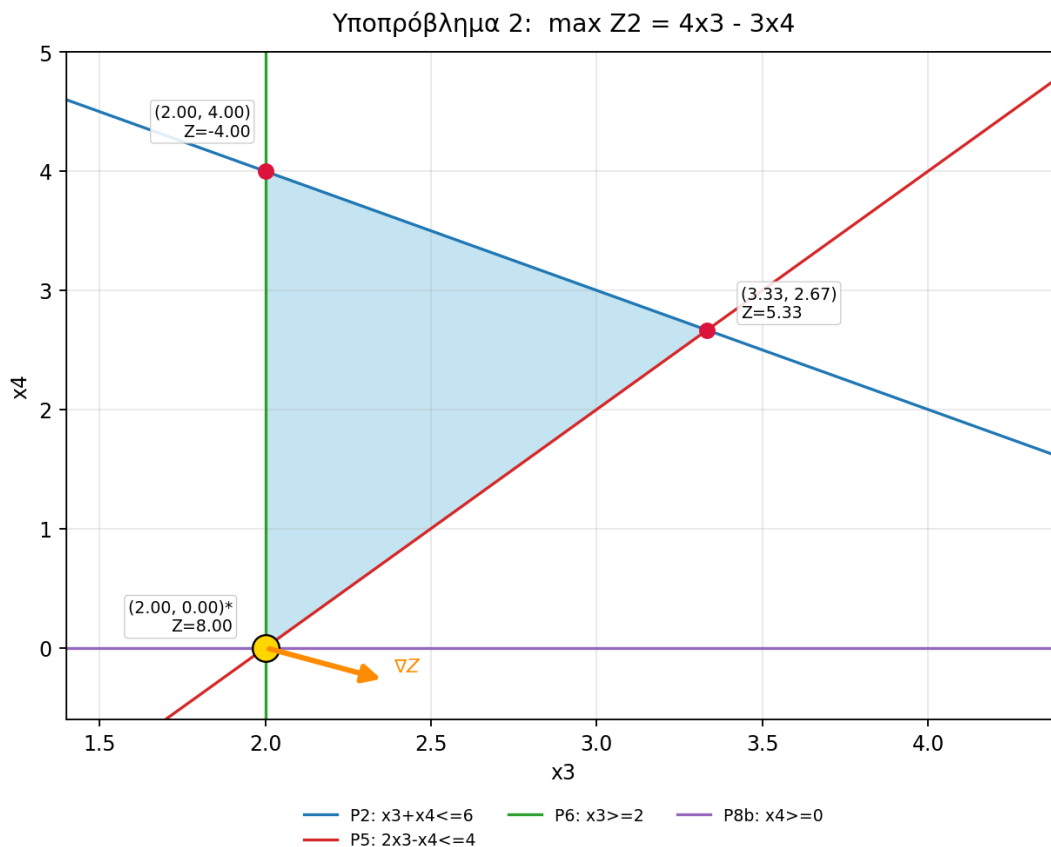
$$K_4 : \begin{cases} x_1 + 2x_2 = 6 \\ 4x_1 + 5x_2 = 20 \end{cases} \Rightarrow 4(6 - 2x_2) + 5x_2 = 20 \Rightarrow x_2 = \frac{4}{3}, x_1 = \frac{10}{3};$$

$$K_5 : 4x_1 + 5 = 20 \Rightarrow x_1 = \frac{15}{4}.$$

$$(x_1^*, x_2^*) = (2, 2) \text{ με } Z_1^* = 3(2) + 7(2) = \mathbf{20}.$$

1.3 Υποπρόβλημα 2: μεταβλητές x_3, x_4

$$\max Z_2 = 4x_3 - 3x_4 \quad \text{με} \quad x_3 + x_4 \leq 6, \quad 2x_3 - x_4 \leq 4, \quad x_3 \geq 2, \quad x_4 \geq 0.$$



Σχήμα 2: Υποπρόβλημα 2. Η βέλτιστη κορυφή (2, 0) είναι εκφυλισμένη (τρεις ευθείες διέρχονται από αυτήν).

Πίνακας 2: Κορυφές του υποπροβλήματος 2.

#	x_3	x_4	Ενεργοί περιορισμοί	πλήθος	Z_2
1	2	0	$\Pi_5, \Pi_6, x_4 = 0$	3	8
2	2	4	Π_2, Π_6	2	-4
3	$\frac{10}{3}$	$\frac{8}{3}$	Π_2, Π_5	2	$\frac{16}{3} \approx 5.33$

Η βέλτιστη κορυφή του υποπροβλήματος 2 είναι εκφυλισμένη. Στο σημείο $(2, 0)$ ισχύουν ταυτόχρονα $x_3 = 2$ (ο Π_6 με ισότητα), $x_4 = 0$ και $2x_3 - x_4 = 2(2) - 0 = 4$ (ο Π_5 με ισότητα). Τρεις ευθείες, δύο διαστάσεις: πρόκειται για εκφυλισμένη κορυφή. Το γεγονός αυτό μεταφέρεται και στο αρχικό πρόβλημα (βλ. επόμενη παράγραφο).

$$(x_3^*, x_4^*) = (2, 0) \text{ με } Z_2^* = 4(2) - 3(0) = 8.$$

1.4 Συνολική λύση, μοναδικότητα και εκφυλισμός

$$x^* = (x_1^*, x_2^*, x_3^*, x_4^*) = (2, 2, 2, 0), \quad Z^* = 3(2) + 7(2) + 4(2) - 3(0) = 28.$$

Έλεγχος όλων των περιορισμών.

$$\begin{array}{ll} (\Pi_1) : 2 + 2(2) = 6 \leq 6 \checkmark \text{ (δεσμευτικός)} & (\Pi_5) : 2(2) - 0 = 4 \leq 4 \checkmark \text{ (δεσμευτικός)} \\ (\Pi_2) : 2 + 0 = 2 \leq 6 \checkmark & (\Pi_6) : x_3 = 2 \geq 2 \checkmark \text{ (δεσμευτικός)} \\ (\Pi_3) : 4(2) + 5(2) = 18 \leq 20 \checkmark & (\Pi_7) : 1 \leq x_2 = 2 \leq 2 \checkmark \text{ (δεσμευτικός άνω)} \\ (\Pi_4) : -2(2) + 2 = -2 \leq 1 \checkmark & (\Pi_8) : x_1 = 2 \geq 0, x_4 = 0 \geq 0 \checkmark \text{ (} x_4 \text{ δεσμευτικός)} \end{array}$$

Μοναδικότητα. Η βέλτιστη λύση είναι μοναδική. Αρκεί να δείξουμε ότι σε κάθε ακμή που ξεκινά από τη βέλτιστη κορυφή η Z γνησίως μειώνεται.

- Υποπρόβλημα 1, κορυφή $(2, 2)$: οι δύο γειτονικές ακμές έχουν κατευθύνσεις $d_a = (-1, 0)$ (κατά μήκος του $x_2 = 2$, προς την $(\frac{1}{2}, 2)$) και $d_b = (2, -1)$ (κατά μήκος του Π_1 , προς την $(\frac{10}{3}, \frac{4}{3})$). Τότε $\nabla Z_1 \cdot d_a = (3, 7) \cdot (-1, 0) = -3 < 0$ και $\nabla Z_1 \cdot d_b = (3, 7) \cdot (2, -1) = 6 - 7 = -1 < 0$.
- Υποπρόβλημα 2, κορυφή $(2, 0)$: οι ακμές έχουν κατευθύνσεις $d_c = (0, 1)$ (κατά μήκος του $x_3 = 2$) και $d_d = (1, 2)$ (κατά μήκος του Π_5). Τότε $\nabla Z_2 \cdot d_c = (4, -3) \cdot (0, 1) = -3 < 0$ και $\nabla Z_2 \cdot d_d = (4, -3) \cdot (1, 2) = 4 - 6 = -2 < 0$.

Επειδή σε όλες τις κατευθύνσεις η Z μειώνεται γνησίως, δεν υπάρχει ακμή βελτίστων και η λύση είναι μοναδική.

Εκφυλισμός στο αρχικό (τετραδιάστατο) πρόβλημα. Στον $\mathbb{R}^4$ μια μη εκφυλισμένη κορυφή απαιτεί ακριβώς 4 ενεργά υπερεπίπεδα. Στο $x^* = (2, 2, 2, 0)$ είναι ενεργά τα $\Pi_1, \Pi_5, \Pi_6, x_2 = 2, x_4 = 0$, δηλαδή **πέντε**. Άρα η βέλτιστη κορυφή του αρχικού προβλήματος είναι **εκφυλισμένη** — ακριβώς όπως προέκυψε και από το υποπρόβλημα 2.

2 Μοντελοποίηση και γραφική επίλυση προβλήματος χαρτοφυλακίου

2.1 (α) Μοντελοποίηση

Μεταβλητές απόφασης.

x_D = ποσό (εκατ. €) σε εγχώριες μετοχές, x_F = ποσό (εκατ. €) σε ξένες μετοχές.

Επιλέγονται *συνεχείς* μεταβλητές γιατί το επενδύόμενο ποσό είναι διαιρετό μέγεθος.

Αντικειμενική συνάρτηση. Η ετήσια απόδοση είναι 11% για τις εγχώριες και 17% για τις ξένες:

$$\max Z = 0.11x_D + 0.17x_F \quad (\text{εκατ. € ετήσια απόδοση}).$$

Περιορισμοί.

(Π ₁) $x_D + x_F \leq 12$	συνολικό διαθέσιμο κεφάλαιο
(Π ₂) $x_D \leq 10$	κανονιστικό όριο εγχώριων
(Π ₃) $x_F \leq 7$	κανονιστικό όριο ξένων
(Π ₄) $x_F \geq \frac{1}{2}x_D \Leftrightarrow -\frac{1}{2}x_D + x_F \geq 0$	ισορροπία: ξένες $\geq$ μισό των εγχώριων
(Π ₅) $x_D \geq \frac{1}{2}x_F \Leftrightarrow x_D - \frac{1}{2}x_F \geq 0$	ισορροπία: εγχώριες $\geq$ μισό των ξένων
$x_D, x_F \geq 0$	μη αρνητικότητα

Σημειώνεται ότι ο Π₁ γράφεται με $\leq$ και όχι με ισότητα: η διατύπωση «ποσό έως 12 εκατομμύρια» δεν υποχρεώνει τον διαχειριστή να επενδύσει όλο το κεφάλαιο.

2.2 (β) Γραφική επίλυση

Πίνακας 3: Κορυφές της εφικτής περιοχής.

#	x_D	x_F	Ενεργοί περιορισμοί	$Z = 0.11x_D + 0.17x_F$
1	0	0	Π ₄ , Π ₅	0
2	3.5	7	Π ₃ , Π ₅	$0.11(3.5) + 0.17(7) = 1.575$
3	5	7	Π ₁ , Π ₃	$0.11(5) + 0.17(7) = \mathbf{1.740}$
4	8	4	Π ₁ , Π ₄	$0.11(8) + 0.17(4) = 1.560$

Υπολογισμός: η κορυφή 2 από $x_D = \frac{1}{2}x_F$ και $x_F = 7$: η κορυφή 3 από $x_D + x_F = 12$ και $x_F = 7$: η κορυφή 4 από $x_D + x_F = 12$ και $x_F = \frac{1}{2}x_D$, δηλαδή $1.5x_D = 12 \Rightarrow x_D = 8, x_F = 4$.

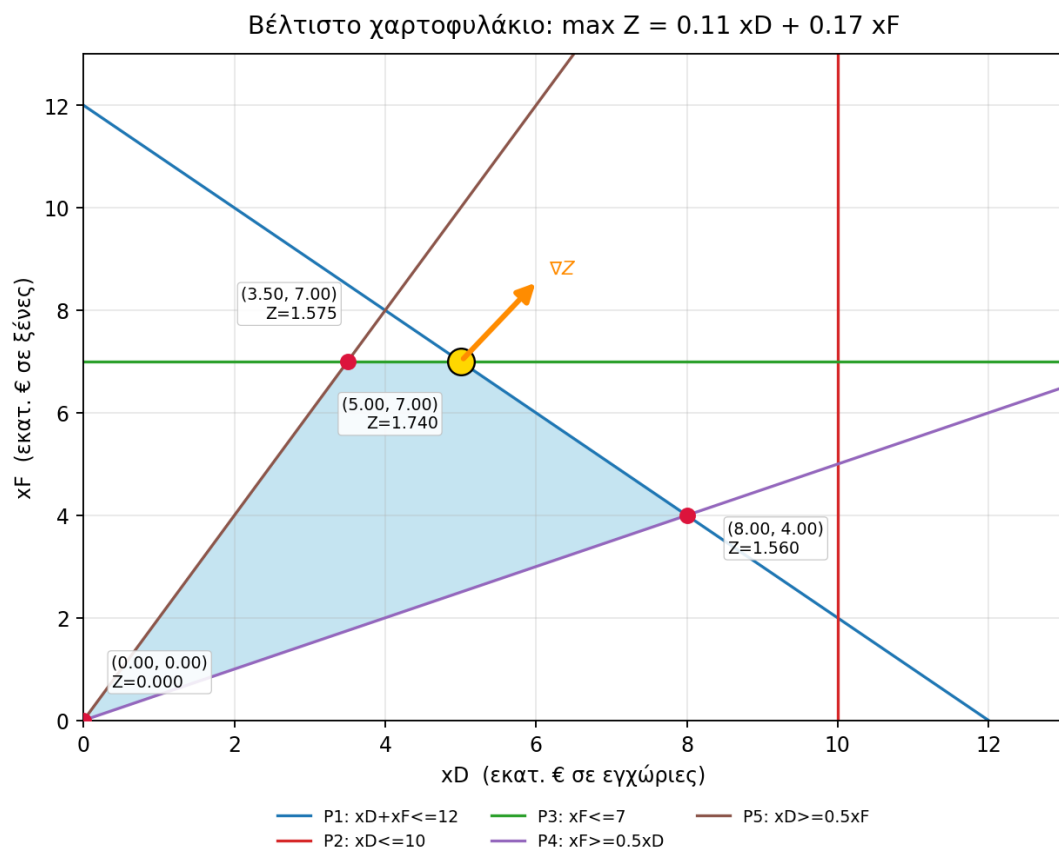
$$x_D^* = 5 \text{ εκατ. €}, \quad x_F^* = 7 \text{ εκατ. €}, \quad Z^* = 1.74 \text{ εκατ. € ετήσια απόδοση (14.5% επί των 12)}.$$

2.3 Πλεονάζων περιορισμός και σκιάδεις τιμές

Ο Π₂ : $x_D \leq 10$ είναι **πλεονάζων (redundant)**. Απόδειξη: από τον Π₄ έχουμε $x_F \geq \frac{1}{2}x_D$, οπότε ο Π₁ δίνει $x_D + \frac{1}{2}x_D \leq x_D + x_F \leq 12$, δηλαδή $x_D \leq 8$. Άρα κάθε σημείο που ικανοποιεί τους Π₁, Π₄ ικανοποιεί αυτόματα και τον Π₂: η ευθεία $x_D = 10$ δεν αγγίζει καθόλου την εφικτή περιοχή. Η αφαίρεσή του δεν μεταβάλλει ούτε την περιοχή ούτε τη λύση.

Οι σκιάδεις τιμές (δυϊκές μεταβλητές) στο βέλτιστο είναι

$$y_{\Pi_1} = 0.11, \quad y_{\Pi_3} = 0.06, \quad y_{\Pi_2} = y_{\Pi_4} = y_{\Pi_5} = 0.$$



Σχήμα 3: Εφικτή περιοχή του προβλήματος χαρτοφυλακίου. Η περιοχή είναι ο «σφήνας» ανάμεσα στις ευθείες $x_F = \frac{1}{2}x_D$ και $x_F = 2x_D$, αποκομμένος από τους $x_F \leq 7$ και $x_D + x_F \leq 12$.

Ερμηνεία:

- $y_{\Pi_1} = 0.11$: ένα επιπλέον εκατομμύριο κεφαλαίου θα αύξανε την απόδοση κατά 0.11 εκατ. €, γιατί το επιπλέον ποσό θα πήγαινε *αναγκαστικά* σε εγχώριες (το όριο των ξένων είναι ήδη εξαντλημένο).
- $y_{\Pi_3} = 0.06 = 0.17 - 0.11$: χαλάρωση του ορίου των ξένων κατά ένα εκατομμύριο επιτρέπει τη μεταφορά ενός εκατομμυρίου από εγχώριες σε ξένες, κερδίζοντας τη διαφορά απόδοσης $17\% - 11\% = 6\%$.
- Οι μηδενικές σκιώδεις τιμές αντιστοιχούν σε μη δεσμευτικούς περιορισμούς — σε συμφωνία με τη συμπληρωματική χαλαρότητα.

3 Μετατροπή προβλήματος με απόλυτες τιμές σε γραμμικό πρόγραμμα

Το πρόβλημα

$$\min Z = |x_1| + |x_2| + |x_3| \quad \text{με} \quad x_1 - 2x_2 = 3, \quad -x_2 + x_3 \leq 1, \quad x_1, x_2, x_3 \in \mathbb{R}.$$

Οι περιορισμοί είναι ήδη γραμμικοί· η μη γραμμικότητα βρίσκεται αποκλειστικά στην αντικειμενική συνάρτηση. Επιπλέον οι μεταβλητές είναι *ελεύθερες*, κάτι που πρέπει επίσης να αντιμετωπιστεί ώστε να προκύψει ΓΠ σε τυπική μορφή.

3.1 Μέθοδος Α: διάσπαση μεταβλητών

Θέτουμε για κάθε j

$$x_j = x_j^+ - x_j^-, \quad x_j^+, x_j^- \geq 0,$$

και προτείνουμε $|x_j| = x_j^+ + x_j^-$. Το πρόβλημα γίνεται

$$\begin{aligned} \min Z &= x_1^+ + x_1^- + x_2^+ + x_2^- + x_3^+ + x_3^- \\ \text{με} \quad &x_1^+ - x_1^- - 2x_2^+ + 2x_2^- = 3 \\ &-x_2^+ + x_2^- + x_3^+ - x_3^- \leq 1 \\ &x_j^+, x_j^- \geq 0, \quad j = 1, 2, 3. \end{aligned}$$

Απόδειξη ισοδυναμίας. Η σχέση $|x_j| = x_j^+ + x_j^-$ δεν ισχύει για αυθαίρετα $x_j^+ \geq 0$ με $x_j^+ - x_j^- = x_j$: ισχύει μόνο όταν $x_j^+ x_j^- = 0$. Πάντα όμως ισχύει $x_j^+ + x_j^- \geq |x_j|$, άρα το νέο ΓΠ είναι *χαλάρωση* προς τα πάνω της αντικειμενικής. Αρκεί λοιπόν να δείξουμε ότι σε κάθε βέλτιστη λύση ισχύει $x_j^+ x_j^- = 0$. Δύο ανεξάρτητα επιχειρήματα:

1. *Επιχείρημα βελτιστότητας:* αν σε κάποια λύση ήταν $x_j^+ > 0$ και $x_j^- > 0$, θέτοντας $\hat{x}_j^\pm = x_j^\pm - \min(x_j^+, x_j^-)$ η διαφορά $x_j^+ - x_j^-$ δεν αλλάζει (άρα η νέα λύση παραμένει εφικτή), ενώ η αντικειμενική *μειώνεται* κατά $2 \min(x_j^+, x_j^-) > 0$ — άτοπο για βέλτιστη λύση. Το επιχείρημα στηρίζεται στο ότι οι συντελεστές των $x_j^\pm$ στην αντικειμενική είναι *θετικοί* και *ίσοι*.
2. *Επιχείρημα βάσης:* οι στήλες των x_j^+ και x_j^- στον πίνακα περιορισμών είναι αντίθετες, άρα γραμμικά εξαρτημένες· δεν μπορούν να ανήκουν ταυτόχρονα σε καμία βάση. Επομένως σε κάθε *βασική* λύση — και ο Simplex επιστρέφει βασική λύση — τουλάχιστον μία από τις δύο είναι μη βασική, δηλαδή μηδέν.

Προσοχή: αν κάποιος συντελεστής της αντικειμενικής ήταν αρνητικός, η μέθοδος θα απέτυχε (το ΓΠ θα γινόταν μη φραγμένο). Εδώ όλοι οι συντελεστές είναι $+1$, οπότε η μετατροπή είναι *έγκυρη*.

3.2 Μέθοδος Β: μετασχηματισμός με άνω φράγματα

Εναλλακτικά εισάγουμε βοηθητικές μεταβλητές $t_j \geq 0$ με $t_j \geq |x_j|$:

$$\min \sum_{j=1}^3 t_j \quad \text{με} \quad t_j \geq x_j, \quad t_j \geq -x_j \quad (j = 1, 2, 3), \quad x_1 - 2x_2 = 3, \quad -x_2 + x_3 \leq 1,$$

με x_j *ελεύθερες* και $t_j \geq 0$. Επειδή ελαχιστοποιούμε τα t_j με θετικούς συντελεστές, σε κάθε βέλτιστη λύση $t_j = \max(x_j, -x_j) = |x_j|$. Η μέθοδος αυτή έχει το πλεονέκτημα ότι δεν διπλασιάζει τον αριθμό των μεταβλητών του αρχικού προβλήματος (προσθέτει 3 μεταβλητές και 6 περιορισμούς αντί για 3 επιπλέον μεταβλητές χωρίς περιορισμούς).

3.3 Λύση και σύνολο βέλτιστων λύσεων

Και οι δύο διατυπώσεις δίνουν $Z^* = 2$. Ωστόσο ο κάθε επιλυτής επιστρέφει *διαφορετική* βέλτιστη λύση:

$$\text{Μέθοδος A} \Rightarrow x = (0, -\frac{3}{2}, -\frac{1}{2}), \quad \text{Μέθοδος B} \Rightarrow x = (1, -1, 0).$$

Αυτό δεν είναι αντίφαση: το πρόβλημα έχει **άπειρες βέλτιστες λύσεις**.

Αναλυτική εύρεση όλου του βέλτιστου συνόλου. Από τον πρώτο περιορισμό, $x_1 = 3 + 2x_2$. Δεδομένου του x_2 , η βέλτιστη επιλογή του x_3 ελαχιστοποιεί το $|x_3|$ υπό τον περιορισμό $x_3 \leq 1 + x_2$, δηλαδή $x_3 = \min(0, 1 + x_2)$. Άρα

$$Z(x_2) = |3 + 2x_2| + |x_2| + \max(0, -(1 + x_2)) = \begin{cases} 3 + 3x_2, & x_2 \geq 0 \\ 3 + x_2, & -1 \leq x_2 < 0 \\ 2, & -\frac{3}{2} \leq x_2 < -1 \\ -4 - 4x_2, & x_2 < -\frac{3}{2}. \end{cases}$$

Η Z είναι κυρτή, φθίνουσα έως το $x_2 = -1$, σταθερή ίση με 2 στο $[-\frac{3}{2}, -1]$ και αύξουσα μετά. Άρα

$$Z^* = 2, \quad X^* = \left\{ (3 + 2x_2, x_2, 1 + x_2) : -\frac{3}{2} \leq x_2 \leq -1 \right\},$$

δηλαδή το βέλτιστο σύνολο είναι η **ακμή** που ενώνει τις κορυφές $(1, -1, 0)$ και $(0, -\frac{3}{2}, -\frac{1}{2})$.

Έλεγχος και των δύο κορυφών.

$$\begin{aligned} x = (1, -1, 0) : \quad & 1 - 2(-1) = 3 \checkmark, \quad -(-1) + 0 = 1 \leq 1 \checkmark, \quad |1| + |-1| + |0| = 2 \checkmark \\ x = (0, -\frac{3}{2}, -\frac{1}{2}) : \quad & 0 - 2(-\frac{3}{2}) = 3 \checkmark, \quad \frac{3}{2} - \frac{1}{2} = 1 \leq 1 \checkmark, \quad 0 + \frac{3}{2} + \frac{1}{2} = 2 \checkmark \end{aligned}$$

Και στις δύο περιπτώσεις και οι δύο περιορισμοί είναι δεσμευτικοί.

4 Μοντελοποίηση προβλήματος δρομολόγησης MMM

Παράμετροι

K	πλήθος διαδρομών μονής κατεύθυνσης, $k = 1, \dots, K$
R	πλήθος τύπων οχημάτων, $r = 1, \dots, R$
d_k	εκτιμώμενη ζήτηση (επιβάτες) στη διαδρομή k
f_{rk}	επιβάτες που εξυπηρετεί ένα όχημα τύπου r στη διαδρομή k
c_{rk}	κόστος ενός οχήματος τύπου r στη διαδρομή k
g_k	έσοδο ανά επιβάτη στη διαδρομή k
v_r	διαθέσιμα οχήματα τύπου r

4.1 (α) Πλήρης κάλυψη της εκτιμώμενης ζήτησης

Μεταβλητές απόφασης. $x_{rk} \in \mathbb{Z}_{\geq 0}$: πλήθος οχημάτων τύπου r που ανατίθενται στη διαδρομή k . Είναι *ακέραιες* γιατί τα οχήματα είναι αδιαίρετα.

Αντικειμενική συνάρτηση. Καθαρό κέρδος = έσοδα − κόστος:

$$\max Z_1 = \sum_{k=1}^K g_k d_k - \sum_{k=1}^K \sum_{r=1}^R c_{rk} x_{rk}.$$

Περιορισμοί.

$$\sum_{r=1}^R f_{rk} x_{rk} \geq d_k, \quad k = 1, \dots, K \quad (\text{πλήρης κάλυψη ζήτησης}) \quad (1)$$

$$\sum_{k=1}^K x_{rk} \leq v_r, \quad r = 1, \dots, R \quad (\text{διαθέσιμος στόλος}) \quad (2)$$

$$x_{rk} \in \mathbb{Z}_{\geq 0}, \quad \forall r, k \quad (\text{ακεραιότητα}) \quad (3)$$

Παρατηρήσεις τεκμηρίωσης.

1. *Γιατί $\geq d_k$ και όχι $= d_k$:* επειδή τα x_{rk} είναι ακέραια, η συνολική χωρητικότητα $\sum_r f_{rk} x_{rk}$ παίρνει διακριτές τιμές και εν γένει δεν μπορεί να ισούται ακριβώς με d_k . Η απαίτηση ισότητας θα καθιστούσε το πρόβλημα τεχνητά μη εφικτό. Η ανισότητα $\geq$ εκφράζει σωστά το «να εξυπηρετηθεί στο ακέραιο η ζήτηση».
2. *Ο πρώτος όρος της αντικειμενικής είναι σταθερά:* αφού κάθε επιβάτης εξυπηρετείται, τα έσοδα $\sum_k g_k d_k$ δεν εξαρτώνται από τις μεταβλητές. Άρα το (α) είναι *ισοδύναμο* με το πρόβλημα $\min \sum_k \sum_r c_{rk} x_{rk}$ — και αυτό εξηγεί γιατί ο περιορισμός $\geq d_k$ δεν οδηγεί σε υπερβολική χωρητικότητα: κάθε περιττό όχημα απλώς αυξάνει το κόστος.
3. *Εφικτότητα:* το μοντέλο είναι εφικτό μόνο αν ο στόλος επαρκεί. Αναγκαία (όχι ικανή) συνθήκη είναι $\sum_r f_{rk} v_r \geq d_k$ για κάθε k : επειδή όμως ο στόλος μοιράζεται ανάμεσα στις διαδρομές, η πραγματική συνθήκη εφικτότητας προκύπτει μόνο από την επίλυση. Αν το πρόβλημα βγει μη εφικτό, η επιχείρηση πρέπει είτε να αυξήσει τον στόλο είτε να δεχτεί μερική κάλυψη (δηλαδή να περάσει στο μοντέλο (β)).
4. Πρόκειται για *ακέραιο* πρόγραμμα (IP), όχι για ΓΠ.

4.2 (β) Χωρητικότητα στο [95%, 105%] της ζήτησης

Τώρα η χωρητικότητα μπορεί να υπολείπεται της ζήτησης, οπότε τα έσοδα δεν είναι πια σταθερά: ένας επιβάτης πληρώνει μόνο αν βρει θέση.

Νέα μεταβλητή. $y_k \geq 0$: πλήθος επιβατών που *πράγματι* εξυπηρετούνται στη διαδρομή k .

$$\max Z_2 = \sum_{k=1}^K g_k y_k - \sum_{k=1}^K \sum_{r=1}^R c_{rk} x_{rk} \quad (4)$$

$$\text{με } 0.95 d_k \leq \sum_{r=1}^R f_{rk} x_{rk} \leq 1.05 d_k, \quad k = 1, \dots, K \quad (5)$$

$$y_k \leq d_k, \quad k = 1, \dots, K \quad (6)$$

$$y_k \leq \sum_{r=1}^R f_{rk} x_{rk}, \quad k = 1, \dots, K \quad (7)$$

$$\sum_{k=1}^K x_{rk} \leq v_r, \quad r = 1, \dots, R; \quad x_{rk} \in \mathbb{Z}_{\geq 0}, \quad y_k \geq 0. \quad (8)$$

Γιατί οι (7)–(8) υλοποιούν σωστά το $y_k = \min\{d_k, \text{χωρητικότητα}\}$. Οι δύο ανισότητες δίνουν μόνο $y_k \leq \min\{\cdot, \cdot\}$. Η ισότητα εξασφαλίζεται από τη βελτιστοποίηση: ο συντελεστής του y_k στην αντικειμενική είναι $g_k > 0$ και μεγιστοποιούμε, ενώ το y_k δεν εμφανίζεται σε κανέναν άλλο περιορισμό. Άρα σε κάθε βέλτιστη λύση το y_k «σπρώχνεται» στο άνω φράγμα του, δηλαδή $y_k = \min\{d_k, \sum_r f_{rk} x_{rk}\}$. Αυτή είναι η τυπική γραμμικοποίηση του $\min$ σε πρόβλημα μεγιστοποίησης — και θα ήταν *λανθασμένη* αν το g_k ήταν αρνητικό ή αν το y_k εμφανιζόταν αλλού. Εναλλακτικά, αν θέλουμε το y_k να είναι ακέραιο (πλήθος επιβατών), προσθέτουμε $y_k \in \mathbb{Z}_{\geq 0}$: η λύση δεν αλλάζει ουσιαστικά αφού τα d_k και f_{rk} είναι ακέραια.

4.3 Αριθμητική επαλήθευση των δύο μοντέλων

Για να επιβεβαιωθεί ότι οι διατυπώσεις είναι σωστές, λύθηκαν σε ένα μικρό στιγμιότυπο ($K = 3$ διαδρομές, $R = 2$ τύποι οχημάτων) με τον MILP επιλυτή της `scipy` (βλ. Παράρτημα, `ex_4.py`).

	διαδρομή 1	διαδρομή 2	διαδρομή 3
ζήτηση d_k	500	300	420
<i>Μοντέλο (α)</i>			
χωρητικότητα	520	300	445 (όλες $\geq d_k$ ✓)
κέρδος = $29,900 - 10,040 = 19,860$			
<i>Μοντέλο (β)</i>			
επιτρεπτό διάστημα	[475, 525]	[285, 315]	[399, 441]
χωρητικότητα	520	290	410 (εντός ορίων ✓)
y_k	500	290	410 = $\min(d_k, \text{χωρ.})$ ✓
κέρδος = 19,840			

Παρατηρούμε ότι στο (β) η επιχείρηση αφήνει σκόπιμα ανεξυπηρετήτους επιβάτες στις διαδρομές 2 και 3 (η χωρητικότητα πέφτει κάτω από τη ζήτηση), γιατί το οριακό κόστος ενός επιπλέον οχήματος υπερβαίνει το οριακό έσοδο — ακριβώς τη συμπεριφορά που το μοντέλο οφείλει να επιτρέπει.

5 Βασικές λύσεις και κορυφές του πολυτόπου

Το πρόβλημα

$$\max Z = -2x_1 + x_2 - 4x_3 + 3x_4$$

$$(\Pi_1) \ x_1 + x_2 + 3x_3 + 2x_4 \leq 4, \quad (\Pi_2) \ x_1 - x_3 + x_4 \leq 2, \quad (\Pi_3) \ 2x_1 + x_2 \leq 3, \quad x \geq 0.$$

5.1 (α) Όλες οι τομές υπερεπιπέδων και οι κορυφές του πολυτόπου

Το πολύτοπο ορίζεται από $m + n = 3 + 4 = 7$ υπερεπίπεδα στον $\mathbb{R}^4$ (οι 3 περιορισμοί και τα 4 υπερεπίπεδα μη αρνητικότητας). Μια υποψήφια κορυφή προκύπτει επιλέγοντας $n = 4$ από αυτά και επιβάλλοντάς τα ως ισότητες, δηλαδή υπάρχουν $\binom{7}{4} = 35$ υποψήφιες τομές.

Πίνακας 4: Και οι 35 τομές τεσσάρων υπερεπιπέδων (Άσκηση 5α).

#	Υπερεπίπεδα σε ισότητα	x_1	x_2	x_3	x_4	Z	Χαρακτηρισμός
1	$\Pi_1, \Pi_2, \Pi_3, x_1=0$	0	3	$-\frac{3}{5}$	$\frac{7}{5}$	$\frac{48}{5}$	μη εφικτή
2	$\Pi_1, \Pi_2, \Pi_3, x_2=0$	$\frac{3}{2}$	0	$\frac{3}{10}$	$\frac{4}{5}$	$-\frac{9}{5}$	ΚΟΡΥΦΗ
3	$\Pi_1, \Pi_2, \Pi_3, x_3=0$	1	1	0	1	2	ΚΟΡΥΦΗ
4	$\Pi_1, \Pi_2, \Pi_3, x_4=0$	$\frac{7}{2}$	-4	$\frac{3}{2}$	0	-17	μη εφικτή
5	$\Pi_1, \Pi_2, x_1=0, x_2=0$	0	0	0	2	6	ΚΟΡΥΦΗ
6	$\Pi_1, \Pi_2, x_1=0, x_3=0$	0	0	0	2	6	ΚΟΡΥΦΗ
7	$\Pi_1, \Pi_2, x_1=0, x_4=0$	0	10	-2	0	18	μη εφικτή
8	$\Pi_1, \Pi_2, x_2=0, x_3=0$	0	0	0	2	6	ΚΟΡΥΦΗ
9	$\Pi_1, \Pi_2, x_2=0, x_4=0$	$\frac{5}{2}$	0	$\frac{1}{2}$	0	-7	μη εφικτή
10	$\Pi_1, \Pi_2, x_3=0, x_4=0$	2	2	0	0	-2	μη εφικτή
11	$\Pi_1, \Pi_3, x_1=0, x_2=0$	γραμμικά εξαρτημένα — δεν ορίζεται τομή					
12	$\Pi_1, \Pi_3, x_1=0, x_3=0$	0	3	0	$\frac{1}{2}$	$\frac{9}{2}$	ΚΟΡΥΦΗ
13	$\Pi_1, \Pi_3, x_1=0, x_4=0$	0	3	$\frac{1}{3}$	0	$\frac{5}{3}$	ΚΟΡΥΦΗ
14	$\Pi_1, \Pi_3, x_2=0, x_3=0$	$\frac{3}{2}$	0	0	$\frac{5}{4}$	$\frac{33}{4}$	μη εφικτή
15	$\Pi_1, \Pi_3, x_2=0, x_4=0$	$\frac{3}{2}$	0	$\frac{5}{6}$	0	$-\frac{19}{3}$	ΚΟΡΥΦΗ
16	$\Pi_1, \Pi_3, x_3=0, x_4=0$	-1	5	0	0	7	μη εφικτή
17	$\Pi_1, x_1=0, x_2=0, x_3=0$	0	0	0	2	6	ΚΟΡΥΦΗ
18	$\Pi_1, x_1=0, x_2=0, x_4=0$	0	0	$\frac{4}{3}$	0	$-\frac{16}{3}$	ΚΟΡΥΦΗ
19	$\Pi_1, x_1=0, x_3=0, x_4=0$	0	4	0	0	4	μη εφικτή
20	$\Pi_1, x_2=0, x_3=0, x_4=0$	4	0	0	0	-8	μη εφικτή
21	$\Pi_2, \Pi_3, x_1=0, x_2=0$	γραμμικά εξαρτημένα — δεν ορίζεται τομή					
22	$\Pi_2, \Pi_3, x_1=0, x_3=0$	0	3	0	2	9	μη εφικτή
23	$\Pi_2, \Pi_3, x_1=0, x_4=0$	0	3	-2	0	11	μη εφικτή
24	$\Pi_2, \Pi_3, x_2=0, x_3=0$	$\frac{3}{2}$	0	0	$\frac{1}{2}$	$-\frac{3}{2}$	ΚΟΡΥΦΗ
25	$\Pi_2, \Pi_3, x_2=0, x_4=0$	$\frac{3}{2}$	0	$-\frac{1}{2}$	0	-1	μη εφικτή
26	$\Pi_2, \Pi_3, x_3=0, x_4=0$	2	-1	0	0	-5	μη εφικτή
27	$\Pi_2, x_1=0, x_2=0, x_3=0$	0	0	0	2	6	ΚΟΡΥΦΗ
28	$\Pi_2, x_1=0, x_2=0, x_4=0$	0	0	-2	0	8	μη εφικτή
29	$\Pi_2, x_1=0, x_3=0, x_4=0$	γραμμικά εξαρτημένα — δεν ορίζεται τομή					
30	$\Pi_2, x_2=0, x_3=0, x_4=0$	2	0	0	0	-4	μη εφικτή
31	$\Pi_3, x_1=0, x_2=0, x_3=0$	γραμμικά εξαρτημένα — δεν ορίζεται τομή					
32	$\Pi_3, x_1=0, x_2=0, x_4=0$	γραμμικά εξαρτημένα — δεν ορίζεται τομή					
33	$\Pi_3, x_1=0, x_3=0, x_4=0$	0	3	0	0	3	ΚΟΡΥΦΗ
34	$\Pi_3, x_2=0, x_3=0, x_4=0$	$\frac{3}{2}$	0	0	0	-3	ΚΟΡΥΦΗ
35	$x_1=0, x_2=0, x_3=0, x_4=0$	0	0	0	0	0	ΚΟΡΥΦΗ

Συγκεντρωτικά.

$$35 = \underbrace{5}_{\substack{\text{γραμμικά εξαρτημένα} \\ (\text{δεν ορίζεται τομή})}} + \underbrace{15}_{\text{τομές μη εφικτές}} + \underbrace{15}_{\text{τομές εφικτές}}.$$

Οι 15 εφικτές τομές αντιστοιχούν σε **11 διακριτά σημεία**, δηλαδή το πολύτοπο έχει 11 κορυφές.

Πίνακας 5: Οι 11 κορυφές του πολυτόπου και τα ενεργά υπερεπίπεδα σε καθεμία.

Κορυφή (x_1, x_2, x_3, x_4)	# ενεργών	Ενεργά υπερεπίπεδα	Εκφυλισμένη;	Z
$(0, 0, 0, 0)$	4	$x_1=x_2=x_3=x_4=0$	όχι	0
$(\frac{3}{2}, 0, 0, 0)$	4	Π_3, x_2, x_3, x_4	όχι	-3
$(0, 3, 0, 0)$	4	Π_3, x_1, x_3, x_4	όχι	3
$(0, 0, \frac{4}{3}, 0)$	4	Π_1, x_1, x_2, x_4	όχι	$-\frac{16}{3}$
$(\frac{3}{2}, 0, \frac{5}{6}, 0)$	4	Π_1, Π_3, x_2, x_4	όχι	$-\frac{19}{3}$
$(0, 3, \frac{1}{3}, 0)$	4	Π_1, Π_3, x_1, x_4	όχι	$\frac{5}{3}$
$(\frac{3}{2}, 0, 0, \frac{1}{2})$	4	Π_2, Π_3, x_2, x_3	όχι	$-\frac{3}{2}$
$(0, 3, 0, \frac{1}{2})$	4	Π_1, Π_3, x_1, x_3	όχι	$\frac{9}{2}$
$(1, 1, 0, 1)$	4	Π_1, Π_2, Π_3, x_3	όχι	2
$(\frac{3}{2}, 0, \frac{3}{10}, \frac{4}{5})$	4	Π_1, Π_2, Π_3, x_2	όχι	$-\frac{9}{5}$
$(0, 0, 0, 2)$	5	$\Pi_1, \Pi_2, x_1, x_2, x_3$	ΝΑΙ	6

Εκφυλισμένη κορυφή. Μόνο η $(0, 0, 0, 2)$ είναι εκφυλισμένη: από αυτήν διέρχονται 5 υπερεπίπεδα αντί για 4. Συγκεκριμένα, οι Π_1 και Π_2 είναι δεσμευτικοί ($0+0+0+2 \cdot 2 = 4$ και $0-0+2 = 2$) και επιπλέον $x_1 = x_2 = x_3 = 0$. Επιλέγοντας οποιαδήποτε 4 από αυτά τα 5 παίρνουμε $\binom{5}{4} = 5$ διαφορετικές περιγραφές του ίδιου σημείου — γι' αυτό οι 15 εφικτές τομές δίνουν μόνο 11 διακριτές κορυφές: $15 = 10 \cdot 1 + 1 \cdot 5$.

5.2 (β) Όλες οι βασικές λύσεις του συστήματος εξισώσεων

Εισάγουμε μεταβλητές χαλάρωσης $s_1, s_2, s_3 \geq 0$:

$$\begin{array}{rrrrrrrr} x_1 + x_2 + 3x_3 + 2x_4 + s_1 & = & 4 \\ x_1 & - & x_3 + x_4 & + & s_2 & = & 2 \\ 2x_1 + x_2 & & & + & s_3 & = & 3 \end{array}$$

Έχουμε 7 μεταβλητές και $m = 3$ εξισώσεις. Μια βασική λύση προκύπτει επιλέγοντας 3 βασικές μεταβλητές (και μηδενίζοντας τις υπόλοιπες 4), δηλαδή υπάρχουν $\binom{7}{3} = 35$ επιλογές βάσης — σε πλήρη αντιστοιχία με τις 35 τομές του ερωτήματος (α).

Πίνακας 6: Και οι 35 επιλογές βάσης και οι αντίστοιχες βασικές λύσεις (Άσκηση 5β).

#	Βάση	x_1	x_2	x_3	x_4	s_1	s_2	s_3	Z	Χαρακτηρισμός
1	$\{x_1, x_2, x_3\}$	$\frac{7}{2}$	-4	$\frac{3}{2}$	0	0	0	0	-17	μη εφικτή
2	$\{x_1, x_2, x_4\}$	1	1	0	1	0	0	0	2	ΒΕΦΛ
3	$\{x_1, x_2, s_1\}$	2	-1	0	0	3	0	0	-5	μη εφικτή
4	$\{x_1, x_2, s_2\}$	-1	5	0	0	0	3	0	7	μη εφικτή
5	$\{x_1, x_2, s_3\}$	2	2	0	0	0	0	-3	-2	μη εφικτή
6	$\{x_1, x_3, x_4\}$	$\frac{3}{2}$	0	$\frac{3}{10}$	$\frac{4}{5}$	0	0	0	$-\frac{9}{5}$	ΒΕΦΛ
7	$\{x_1, x_3, s_1\}$	0	$-\frac{1}{2}$	0	4	0	0	0	-1	μη εφικτή
8	$\{x_1, x_3, s_2\}$	0	$\frac{5}{6}$	0	0	0	$\frac{4}{3}$	0	$-\frac{19}{3}$	ΒΕΦΛ
9	$\{x_1, x_3, s_3\}$	0	$\frac{1}{2}$	0	0	0	0	-2	-7	μη εφικτή
10	$\{x_1, x_4, s_1\}$	0	0	$\frac{1}{2}$	$\frac{3}{2}$	0	0	0	$-\frac{3}{2}$	ΒΕΦΛ
11	$\{x_1, x_4, s_2\}$	$\frac{3}{2}$	0	0	$\frac{5}{4}$	0	$-\frac{3}{4}$	0	$\frac{3}{4}$	μη εφικτή
12	$\{x_1, x_4, s_3\}$	0	0	0	2	0	0	3	6	ΒΕΦΛ εκφ.
13	$\{x_1, s_1, s_2\}$	$\frac{3}{2}$	0	0	0	$\frac{5}{2}$	$\frac{1}{2}$	0	-3	ΒΕΦΛ
14	$\{x_1, s_1, s_3\}$	2	0	0	0	2	0	-1	-4	μη εφικτή
15	$\{x_1, s_2, s_3\}$	4	0	0	0	0	-2	-5	-8	μη εφικτή
16	$\{x_2, x_3, x_4\}$	0	3	$-\frac{3}{5}$	$\frac{7}{5}$	0	0	0	$\frac{48}{5}$	μη εφικτή
17	$\{x_2, x_3, s_1\}$	0	3	-2	0	7	0	0	11	μη εφικτή
18	$\{x_2, x_3, s_2\}$	0	3	$\frac{1}{3}$	0	0	$\frac{7}{3}$	0	$\frac{5}{3}$	ΒΕΦΛ

#	Βάση	x_1	x_2	x_3	x_4	s_1	s_2	s_3	Z	Χαρακτηρισμός
19	$\{x_2, x_3, s_3\}$	0	10	-2	0	0	0	-7	18	μη εφικτή
20	$\{x_2, x_4, s_1\}$	0	3	0	2	-3	0	0	9	μη εφικτή
21	$\{x_2, x_4, s_2\}$	0	3	0	$\frac{1}{2}$	0	$\frac{3}{2}$	0	$\frac{9}{2}$	ΒΕΦΛ
22	$\{x_2, x_4, s_3\}$	0	0	0	2	0	0	3	6	ΒΕΦΛ εκφ.
23	$\{x_2, s_1, s_2\}$	0	3	0	0	1	2	0	3	ΒΕΦΛ
24	$\{x_2, s_1, s_3\}$			ο B είναι ιδιάζων — δεν αποτελεί βάση						
25	$\{x_2, s_2, s_3\}$	0	4	0	0	0	2	-1	4	μη εφικτή
26	$\{x_3, x_4, s_1\}$			ο B είναι ιδιάζων — δεν αποτελεί βάση						
27	$\{x_3, x_4, s_2\}$			ο B είναι ιδιάζων — δεν αποτελεί βάση						
28	$\{x_3, x_4, s_3\}$	0	0	0	2	0	0	3	6	ΒΕΦΛ εκφ.
29	$\{x_3, s_1, s_2\}$			ο B είναι ιδιάζων — δεν αποτελεί βάση						
30	$\{x_3, s_1, s_3\}$	0	0	-2	0	10	0	3	8	μη εφικτή
31	$\{x_3, s_2, s_3\}$	0	0	$\frac{4}{3}$	0	0	$\frac{10}{3}$	3	$-\frac{16}{3}$	ΒΕΦΛ
32	$\{x_4, s_1, s_2\}$			ο B είναι ιδιάζων — δεν αποτελεί βάση						
33	$\{x_4, s_1, s_3\}$	0	0	0	2	0	0	3	6	ΒΕΦΛ εκφ.
34	$\{x_4, s_2, s_3\}$	0	0	0	2	0	0	3	6	ΒΕΦΛ εκφ.
35	$\{s_1, s_2, s_3\}$	0	0	0	0	4	2	3	0	ΒΕΦΛ

Συγκεντρωτικά.

$$35 = \underbrace{5}_{\text{ιδιάζων } B} + \underbrace{15}_{\text{μη εφικτές βασικές λύσεις}} + \underbrace{15}_{\text{ΒΕΦΛ}}, \quad \text{εκ των οποίων 5 εκφυλισμένες.}$$

Οι 5 εκφυλισμένες ΒΕΦΛ είναι οι υπ' αριθμόν 12, 22, 28, 33, 34 με βάσεις $\{x_1, x_4, s_3\}$, $\{x_2, x_4, s_3\}$, $\{x_3, x_4, s_3\}$, $\{x_4, s_1, s_3\}$, $\{x_4, s_2, s_3\}$ — και οι πέντε δίνουν το ίδιο σημείο $(0, 0, 0, 2)$.

5.3 (γ) Αντιστοίχιση βασικών λύσεων $\leftrightarrow$ κορυφών

Η αντιστοιχία είναι η εξής: αν η βάση είναι B , οι 4 μη βασικές μεταβλητές μηδενίζονται· καθεμία από αυτές αντιστοιχεί σε ένα ενεργό υπερεπίπεδο ($x_j = 0$ για διαρθρωτική μεταβλητή, Π_i δεσμευτικός για μεταβλητή χαλάρωσης s_i). Άρα κάθε βασική λύση $\leftrightarrow$ μία τομή 4 υπερεπιπέδων, και κάθε εφικτή βασική λύση (ΒΕΦΛ) $\leftrightarrow$ μία κορυφή. Η αντιστοιχία είναι 1-1 μόνο για τις μη εκφυλισμένες κορυφές.

Πίνακας 7: Αντιστοίχιση των 15 ΒΕΦΛ στις 11 κορυφές.

Κορυφή	Βάσεις που της αντιστοιχούν	Z
$(\frac{3}{2}, 0, \frac{5}{6}, 0)$	$\{x_1, x_3, s_2\}$	$-\frac{19}{3}$
$(0, 0, \frac{4}{3}, 0)$	$\{x_3, s_2, s_3\}$	$-\frac{16}{3}$
$(\frac{3}{2}, 0, 0, 0)$	$\{x_1, s_1, s_2\}$	-3
$(\frac{3}{2}, 0, \frac{3}{10}, \frac{4}{5})$	$\{x_1, x_3, x_4\}$	$-\frac{9}{5}$
$(\frac{3}{2}, 0, 0, \frac{1}{2})$	$\{x_1, x_4, s_1\}$	$-\frac{3}{2}$
$(0, 0, 0, 0)$	$\{s_1, s_2, s_3\}$	0
$(0, 3, \frac{1}{3}, 0)$	$\{x_2, x_3, s_2\}$	$\frac{5}{3}$
$(1, 1, 0, 1)$	$\{x_1, x_2, x_4\}$	2
$(0, 3, 0, 0)$	$\{x_2, s_1, s_2\}$	3
$(0, 3, 0, \frac{1}{2})$	$\{x_2, x_4, s_2\}$	$\frac{9}{2}$
$(0, 0, 0, 2)$	$\{x_1, x_4, s_3\}, \{x_2, x_4, s_3\}, \{x_3, x_4, s_3\}, \{x_4, s_1, s_3\}, \{x_4, s_2, s_3\}$	6

Βέλτιστη λύση: $x^* = (0, 0, 0, 2)$, $s^* = (0, 0, 3)$, με $Z^* = 6$.

Η βέλτιστη κορυφή είναι **εκφυλισμένη**: σε κάθε μία από τις 5 βάσεις που την περιγράφουν, μία βασική μεταβλητή έχει τιμή 0 (π.χ. στη βάση $\{x_1, x_4, s_3\}$ είναι $x_1 = 0$). Ο έλεγχος $15 = 10 \cdot 1 + 1 \cdot 5$ επιβεβαιώνει πλήρως την αντιστοιχία μεταξύ άλγεβρας (ερώτημα β) και γεωμετρίας (ερώτημα α).

Πρακτική συνέπεια. Ο εκφυλισμός σημαίνει ότι, φτάνοντας στη βέλτιστη κορυφή, ο Simplex μπορεί να εκτελέσει έως 4 επιπλέον *εκφυλισμένα* pivots (αλλαγές βάσης με λόγο 0) χωρίς καμία βελτίωση του Z — φαινόμενο γνωστό ως *stalling*. Χωρίς κανόνα αποφυγής κύκλων (π.χ. κανόνας Bland) θα ήταν θεωρητικά δυνατό και το *cycling*.

6 Αλγόριθμος Simplex και γράφος γειτνίασης

Το πρόβλημα

$$\max Z = 3x_1 + 5x_2 + 6x_3, \quad \begin{array}{ll} (\Pi_1) & 2x_1 + x_2 + x_3 \leq 4 \\ (\Pi_2) & x_1 + 2x_2 + x_3 \leq 4 \end{array} \quad \begin{array}{ll} (\Pi_3) & x_1 + x_2 + 2x_3 \leq 4 \\ (\Pi_4) & x_1 + x_2 + x_3 \leq 3 \end{array} \quad x \geq 0.$$

Με $s_1, \dots, s_4 \geq 0$ έχουμε 7 μεταβλητές και $m = 4$ εξισώσεις, άρα κάθε βάση έχει 4 μεταβλητές.

6.1 (α) Εκτέλεση του Simplex

Επανάληψη 0 (αρχική ΒΕΦΛ: η αρχή των αξόνων).

Βάση	x_1	x_2	x_3	s_1	s_2	s_3	s_4	RHS
s_1	2	1	1	1	0	0	0	4
s_2	1	2	1	0	1	0	0	4
s_3	1	1	2	0	0	1	0	4
s_4	1	1	1	0	0	0	1	3
$c_j - z_j$	3	5	6	0	0	0	0	$Z = 0$

Εισερχόμενη: x_3 (μεγαλύτερο $c_j - z_j = 6$). Ratio test: $\frac{4}{1} = 4$, $\frac{4}{1} = 4$, $\frac{4}{2} = \mathbf{2}$, $\frac{3}{1} = 3 \Rightarrow$ εξέρχεται η s_3 , στοιχείο οδηγός 2.

Επανάληψη 1.

Βάση	x_1	x_2	x_3	s_1	s_2	s_3	s_4	RHS
s_1	$\frac{3}{2}$	$\frac{1}{2}$	0	1	0	$-\frac{1}{2}$	0	2
s_2	$\frac{1}{2}$	$\frac{3}{2}$	0	0	1	$-\frac{1}{2}$	0	2
x_3	$\frac{1}{2}$	$\frac{1}{2}$	1	0	0	$\frac{1}{2}$	0	2
s_4	$\frac{1}{2}$	$\frac{1}{2}$	0	0	0	$-\frac{1}{2}$	1	1
$c_j - z_j$	0	2	0	0	0	-3	0	$Z = 12$

Εισερχόμενη: x_2 (μοναδικό θετικό $c_j - z_j = 2$). Ratio test: $\frac{2}{1/2} = 4$, $\frac{2}{3/2} = \frac{4}{3}$, $\frac{2}{1/2} = 4$, $\frac{1}{1/2} = 2 \Rightarrow$ εξέρχεται η s_2 , στοιχείο οδηγός $\frac{3}{2}$.

Επανάληψη 2 (τελική).

Βάση	x_1	x_2	x_3	s_1	s_2	s_3	s_4	RHS
s_1	$\frac{4}{3}$	0	0	1	$-\frac{1}{3}$	$-\frac{1}{3}$	0	$\frac{4}{3}$
x_2	$\frac{1}{3}$	1	0	0	$\frac{2}{3}$	$-\frac{1}{3}$	0	$\frac{4}{3}$
x_3	$\frac{1}{3}$	0	1	0	$-\frac{1}{3}$	$\frac{2}{3}$	0	$\frac{4}{3}$
s_4	$\frac{1}{3}$	0	0	0	$-\frac{1}{3}$	$-\frac{1}{3}$	1	$\frac{1}{3}$
$c_j - z_j$	$-\frac{2}{3}$	0	0	0	$-\frac{4}{3}$	$-\frac{7}{3}$	0	$Z = \frac{44}{3}$

Όλα τα $c_j - z_j \leq 0$: ο αλγόριθμος τερματίζει.

$$x^* = \left(0, \frac{4}{3}, \frac{4}{3}\right), \quad Z^* = \frac{44}{3} \approx 14.667.$$

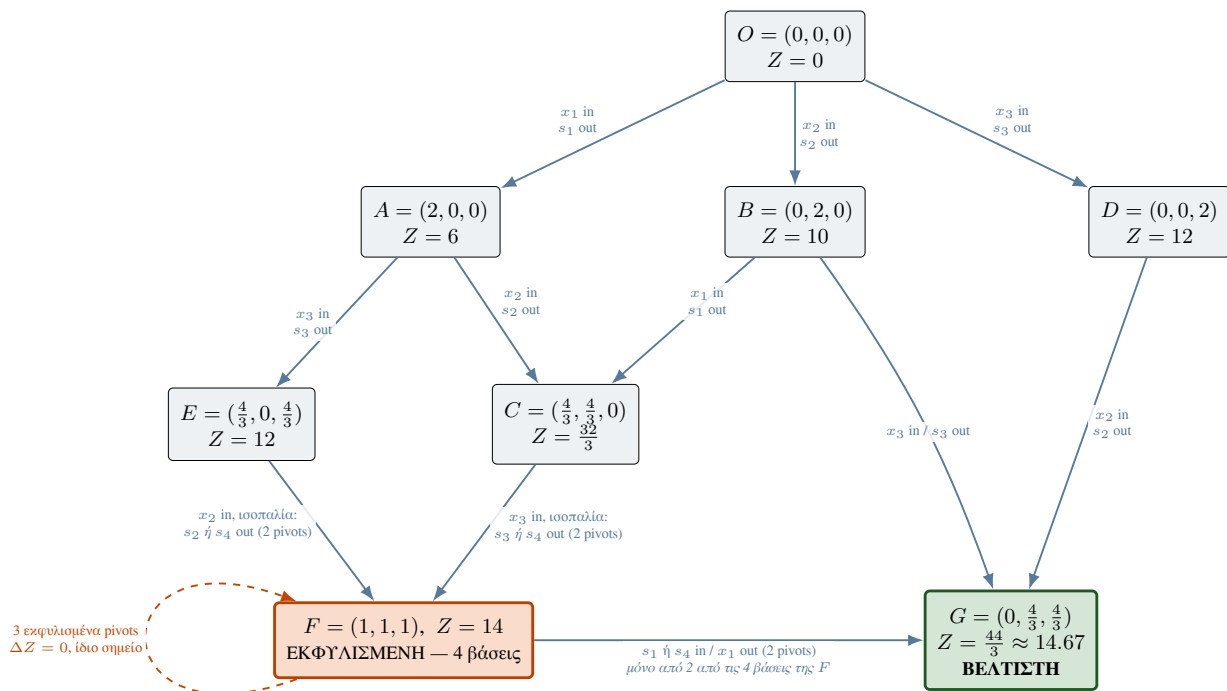
Τα $c_j - z_j$ των μη βασικών x_1, s_2, s_3 είναι γνησίως αρνητικά, άρα η βέλτιστη λύση είναι **μοναδική**.

6.2 (β) Simplex adjacency graph

Στο ερώτημα αυτό διερευνώνται όλες οι εναλλακτικές επιλογές: σε κάθε κόμβο δοκιμάζεται *κάθε* εισερχόμενη μεταβλητή με $c_j - z_j > 0$ και, όταν το ratio test έχει ισοπαλία, *κάθε* δυνατή εξερχόμενη. Η πλήρης απαρίθμηση (κώδικας `ex_6.py`) δίνει **8 κορυφές**, **11 βάσεις** και **17 επιτρεπτά pivots**.

Πίνακας 8: Οι κορυφές του πολυτόπου και οι βάσεις που τις περιγράφουν.

Κόμβος	(x_1, x_2, x_3)	Z	Βάσεις
O	$(0, 0, 0)$	0	$\{s_1, s_2, s_3, s_4\}$
A	$(2, 0, 0)$	6	$\{x_1, s_2, s_3, s_4\}$
B	$(0, 2, 0)$	10	$\{x_2, s_1, s_3, s_4\}$
C	$(\frac{4}{3}, \frac{4}{3}, 0)$	$\frac{32}{3}$	$\{x_1, x_2, s_3, s_4\}$
D	$(0, 0, 2)$	12	$\{x_3, s_1, s_2, s_4\}$
E	$(\frac{4}{3}, 0, \frac{4}{3})$	12	$\{x_1, x_3, s_2, s_4\}$
F	$(1, 1, 1)$	14	$\{x_1, x_2, x_3, s_1\}, \{x_1, x_2, x_3, s_2\}, \{x_1, x_2, x_3, s_3\}, \{x_1, x_2, x_3, s_4\}$
G	$(0, \frac{4}{3}, \frac{4}{3})$	$\frac{44}{3}$	$\{x_2, x_3, s_1, s_4\}$ (βέλτιστη)



Σχήμα 4: Simplex adjacency graph. Κάθε βέλος είναι ένα επιτρεπτό pivot (εισερχόμενη με $c_j - z_j > 0$ και εξερχόμενη από το ratio test). Με πορτοκαλί διακεκομμένο βέλος τα *εκφυλισμένα* pivots στην κορυφή F , όπου αλλάζει η βάση χωρίς να μετακινηθεί το σημείο και χωρίς να αυξηθεί το Z .

Ανάλυση των διαδρομών

- Από την αρχή O υπάρχουν **τρεις** επιλογές εισερχόμενης (x_1, x_2 ή x_3), που οδηγούν αντίστοιχα στις A, B, D .
- Η **συντομότερη** διαδρομή είναι $O \rightarrow D \rightarrow G$: δύο μόνο επαναλήψεις. Αυτή ακολουθεί και ο κανόνας Dantzig του ερωτήματος (α), αφού το x_3 έχει τον μεγαλύτερο συντελεστή.
- Η **μακρύτερη** είναι $O \rightarrow A \rightarrow C \rightarrow F \rightarrow \dots \rightarrow G$, όπου στο F ενδέχεται να χρειαστούν εκφυλισμένα pivots πριν βρεθεί η ακμή προς το G .

- Το πλήθος των βημάτων εξαρτάται μόνο από τους κανόνες επιλογής, όχι από το πρόβλημα: όλες οι διαδρομές καταλήγουν στην ίδια (μοναδική) βέλτιστη κορυφή G .

Η εκφυλισμένη κορυφή $F = (1, 1, 1)$ και τα εκφυλισμένα pivots. Στο σημείο $(1, 1, 1)$ ισχύουν και οι τέσσερις περιορισμοί με ισότητα:

$$2+1+1 = 4, \quad 1+2+1 = 4, \quad 1+1+2 = 4, \quad 1+1+1 = 3,$$

δηλαδή $s_1 = s_2 = s_3 = s_4 = 0$. Στον $\mathbb{R}^3$ μια κορυφή χρειάζεται 3 ενεργά επίπεδα· εδώ τέμνονται 4, άρα η F είναι **εκφυλισμένη**. Αλγεβρικά: η βάση έχει 4 μεταβλητές αλλά μόνο 3 είναι μη μηδενικές ($x_1 = x_2 = x_3 = 1$), οπότε η τέταρτη βασική μεταβλητή είναι μία από τις s_1, s_2, s_3, s_4 με τιμή 0 — εξ ου και οι 4 διαφορετικές βάσεις.

Η πρακτική συνέπεια φαίνεται στον γράφο: από τις 4 βάσεις της F , μόνο δύο ($\{x_1, x_2, x_3, s_1\}$ και $\{x_1, x_2, x_3, s_4\}$) έχουν pivot που οδηγεί στη βέλτιστη G . Αν ο αλγόριθμος φτάσει στην F μέσω της C με τη βάση $\{x_1, x_2, x_3, s_3\}$, πρέπει πρώτα να εκτελέσει ένα ή δύο **εκφυλισμένα** pivots (s_1 ή s_2 εισερχόμενη, με λόγο 0) που δεν μετακινούν το σημείο και αφήνουν το $Z = 14$ αμετάβλητο. Αυτό ακριβώς είναι το φαινόμενο του *stalling*.

Πίνακας 9: Και τα 17 επιτρεπτά pivots του γράφου.

Από (βάση)	Pivot	Προς (βάση)	Z
$O \{s_1, s_2, s_3, s_4\}$	x_1 in / s_1 out	$A \{x_1, s_2, s_3, s_4\}$	$0 \rightarrow 6$
$O \{s_1, s_2, s_3, s_4\}$	x_2 in / s_2 out	$B \{x_2, s_1, s_3, s_4\}$	$0 \rightarrow 10$
$O \{s_1, s_2, s_3, s_4\}$	x_3 in / s_3 out	$D \{x_3, s_1, s_2, s_4\}$	$0 \rightarrow 12$
$A \{x_1, s_2, s_3, s_4\}$	x_2 in / s_2 out	$C \{x_1, x_2, s_3, s_4\}$	$6 \rightarrow \frac{32}{3}$
$A \{x_1, s_2, s_3, s_4\}$	x_3 in / s_3 out	$E \{x_1, x_3, s_2, s_4\}$	$6 \rightarrow 12$
$B \{x_2, s_1, s_3, s_4\}$	x_1 in / s_1 out	$C \{x_1, x_2, s_3, s_4\}$	$10 \rightarrow \frac{32}{3}$
$B \{x_2, s_1, s_3, s_4\}$	x_3 in / s_3 out	$G \{x_2, x_3, s_1, s_4\}$	$10 \rightarrow \frac{44}{3}$
$C \{x_1, x_2, s_3, s_4\}$	x_3 in / s_3 out	$F \{x_1, x_2, x_3, s_4\}$	$\frac{32}{3} \rightarrow 14$
$C \{x_1, x_2, s_3, s_4\}$	x_3 in / s_4 out	$F \{x_1, x_2, x_3, s_3\}$	$\frac{32}{3} \rightarrow 14$
$D \{x_3, s_1, s_2, s_4\}$	x_2 in / s_2 out	$G \{x_2, x_3, s_1, s_4\}$	$12 \rightarrow \frac{44}{3}$
$E \{x_1, x_3, s_2, s_4\}$	x_2 in / s_2 out	$F \{x_1, x_2, x_3, s_4\}$	$12 \rightarrow 14$
$E \{x_1, x_3, s_2, s_4\}$	x_2 in / s_4 out	$F \{x_1, x_2, x_3, s_2\}$	$12 \rightarrow 14$
$F \{x_1, x_2, x_3, s_3\}$	s_1 in / s_3 out	$F \{x_1, x_2, x_3, s_1\}$	$14 \rightarrow 14$ (εκφυλισμένο)
$F \{x_1, x_2, x_3, s_3\}$	s_2 in / s_3 out	$F \{x_1, x_2, x_3, s_2\}$	$14 \rightarrow 14$ (εκφυλισμένο)
$F \{x_1, x_2, x_3, s_2\}$	s_1 in / s_2 out	$F \{x_1, x_2, x_3, s_1\}$	$14 \rightarrow 14$ (εκφυλισμένο)
$F \{x_1, x_2, x_3, s_1\}$	s_4 in / x_1 out	$G \{x_2, x_3, s_1, s_4\}$	$14 \rightarrow \frac{44}{3}$
$F \{x_1, x_2, x_3, s_4\}$	s_1 in / x_1 out	$G \{x_2, x_3, s_1, s_4\}$	$14 \rightarrow \frac{44}{3}$

7 Τεχνική Φάσης I του αλγορίθμου δύο φάσεων

Η Φάση I κατασκευάζει ένα βοηθητικό πρόβλημα με τεχνητές μεταβλητές $A_i \geq 0$ και ελαχιστοποιεί το άθροισμά τους (ισοδύναμα μεγιστοποιεί $W = -\sum_i A_i$). Το αρχικό σύστημα είναι εφικτό αν και μόνο αν $W^* = 0$.

7.1 Σύστημα (α)

$$\begin{aligned} -5x_1 + x_2 - 3x_3 + 3x_4 + 8x_5 &\leq -3 \\ 3x_1 - x_2 + 2x_3 - x_4 - 5x_5 &\leq 2 \quad x \geq 0 \\ -2x_1 + x_2 - x_3 + 3x_5 &\leq 2 \end{aligned}$$

Πολλαπλασιάζουμε την 1η ανισότητα με -1 ώστε το δεξί μέλος να γίνει θετικό: $5x_1 - x_2 + 3x_3 - 3x_4 - 8x_5 \geq 3$. Προσθέτουμε μεταβλητή πλεονάσματος e_1 και τεχνητή A_1 στην πρώτη, και μεταβλητές χαλάρωσης s_2, s_3 στις άλλες δύο:

$$\begin{aligned} 5x_1 - x_2 + 3x_3 - 3x_4 - 8x_5 - e_1 + A_1 &= 3 \\ 3x_1 - x_2 + 2x_3 - x_4 - 5x_5 + s_2 &= 2 \\ -2x_1 + x_2 - x_3 + 3x_5 + s_3 &= 2 \end{aligned}$$

Με $W = -A_1$ και $A_1 = 3 - 5x_1 + x_2 - 3x_3 + 3x_4 + 8x_5 + e_1$ προκύπτει $W + 5x_1 - x_2 + 3x_3 - 3x_4 - 8x_5 - e_1 = -3$, δηλαδή η γραμμή κριτηρίου έχει $c_j - z_j = (5, -1, 3, -3, -8, -1, 0, 0, 0)$.

Επανάληψη 0.

Βάση	x_1	x_2	x_3	x_4	x_5	e_1	s_2	s_3	A_1	RHS
A_1	5	-1	3	-3	-8	-1	0	0	1	3
s_2	3	-1	2	-1	-5	0	1	0	0	2
s_3	-2	1	-1	0	3	0	0	1	0	2
$c_j - z_j$	5	-1	3	-3	-8	-1	0	0	0	$W = -3$

Εισέρχεται η x_1 ($c_j - z_j = 5$). Ratio test: $\frac{3}{5} = \mathbf{0.6}$ και $\frac{2}{3} \approx 0.667$ (η γραμμή s_3 έχει αρνητικό συντελεστή, δεν συμμετέχει). Εξέρχεται η τεχνητή A_1 .

Επανάληψη 1 (τελική).

Βάση	x_1	x_2	x_3	x_4	x_5	e_1	s_2	s_3	A_1	RHS
x_1	1	$-\frac{1}{5}$	$\frac{3}{5}$	$-\frac{3}{5}$	$-\frac{8}{5}$	$-\frac{1}{5}$	0	0	$\frac{1}{5}$	$\frac{3}{5}$
s_2	0	$-\frac{2}{5}$	$\frac{1}{5}$	$\frac{4}{5}$	$-\frac{1}{5}$	$\frac{3}{5}$	1	0	$-\frac{3}{5}$	$\frac{1}{5}$
s_3	0	$\frac{3}{5}$	$\frac{1}{5}$	$-\frac{6}{5}$	$-\frac{1}{5}$	$-\frac{2}{5}$	0	1	$\frac{2}{5}$	$\frac{16}{5}$
$c_j - z_j$	0	0	0	0	0	0	0	0	-1	$W = 0$

$W^* = 0$ και η τεχνητή A_1 έχει βγει από τη βάση $\Rightarrow$ το σύστημα (α) είναι **ΕΦΙΚΤΟ**. Αρχική ΒΕΦΛ για τη Φάση II:

$$x_1 = \frac{3}{5} = 0.6, \quad s_2 = \frac{1}{5} = 0.2, \quad s_3 = \frac{16}{5} = 3.2, \quad x_2 = x_3 = x_4 = x_5 = e_1 = 0,$$

δηλαδή $x = (0.6, 0, 0, 0, 0)$.

Άμεση επαλήθευση: $-5(0.6) = -3 \leq -3 \checkmark, 3(0.6) = 1.8 \leq 2 \checkmark, -2(0.6) = -1.2 \leq 2 \checkmark$.

7.2 Σύστημα (β)

$$\begin{aligned} x_1 + x_2 + x_3 + x_4 + x_5 &= 2 & (E_1) \\ -x_1 + 2x_2 + x_3 - 3x_4 + x_5 &= 1 & (E_2) \quad x \geq 0 \\ x_1 - 3x_2 - 2x_3 + 2x_4 - 2x_5 &= -4 & (E_3) \end{aligned}$$

Πολλαπλασιάζουμε την E_3 με -1 και προσθέτουμε τεχνητές A_1, A_2, A_3 σε κάθε εξίσωση:

$$\begin{aligned} x_1 + x_2 + x_3 + x_4 + x_5 + A_1 &= 2 \\ -x_1 + 2x_2 + x_3 - 3x_4 + x_5 + A_2 &= 1 \\ -x_1 + 3x_2 + 2x_3 - 2x_4 + 2x_5 + A_3 &= 4 \end{aligned}$$

Με $W = -(A_1 + A_2 + A_3)$ και προσθέτοντας τις τρεις εξισώσεις προκύπτει $A_1 + A_2 + A_3 = 7 + x_1 - 6x_2 - 4x_3 + 4x_4 - 4x_5$, άρα $W + x_1 - 6x_2 - 4x_3 + 4x_4 - 4x_5 = -7$.

Επανάληψη 0.

Βάση	x_1	x_2	x_3	x_4	x_5	A_1	A_2	A_3	RHS
A_1	1	1	1	1	1	1	0	0	2
A_2	-1	2	1	-3	1	0	1	0	1
A_3	-1	3	2	-2	2	0	0	1	4
$c_j - z_j$	-1	6	4	-4	4	0	0	0	$W = -7$

Εισέρχεται η x_2 . Ratio test: $\frac{2}{1} = 2, \frac{1}{2} = \mathbf{0.5}, \frac{4}{3} \approx 1.33 \Rightarrow$ εξέρχεται η A_2 .

Επανάληψη 1.

Βάση	x_1	x_2	x_3	x_4	x_5	A_1	A_2	A_3	RHS
A_1	$\frac{3}{2}$	0	$\frac{1}{2}$	$\frac{5}{2}$	$\frac{1}{2}$	1	$-\frac{1}{2}$	0	$\frac{3}{2}$
x_2	$-\frac{1}{2}$	1	$\frac{1}{2}$	$-\frac{3}{2}$	$\frac{1}{2}$	0	$\frac{1}{2}$	0	$\frac{1}{2}$
A_3	$\frac{1}{2}$	0	$\frac{1}{2}$	$\frac{5}{2}$	$\frac{1}{2}$	0	$-\frac{3}{2}$	1	$\frac{5}{2}$
$c_j - z_j$	2	0	1	5	1	0	-3	0	$W = -4$

Εισέρχεται η x_4 . Ratio test: $\frac{3/2}{5/2} = \mathbf{0.6}, \frac{5/2}{5/2} = 1 \Rightarrow$ εξέρχεται η A_1 .

Επανάληψη 2 (τελική).

Βάση	x_1	x_2	x_3	x_4	x_5	A_1	A_2	A_3	RHS
x_4	$\frac{3}{5}$	0	$\frac{1}{5}$	1	$\frac{1}{5}$	$\frac{2}{5}$	$-\frac{1}{5}$	0	$\frac{3}{5}$
x_2	$\frac{2}{5}$	1	$\frac{4}{5}$	0	$\frac{4}{5}$	$\frac{3}{5}$	$\frac{1}{5}$	0	$\frac{7}{5}$
A_3	-1	0	0	0	0	-1	-1	1	1
$c_j - z_j$	-1	0	0	0	0	-2	-2	0	$W = -1$

Όλα τα $c_j - z_j \leq 0$: η Φάση I τερμάτισε με $W^* = -1 < 0$, δηλαδή $A_1 + A_2 + A_3 = 1 > 0$. Η τεχνητή A_3 παραμένει στη βάση με θετική τιμή 1. Άρα το σύστημα (β) είναι **ΜΗ ΕΦΙΚΤΟ**.

Αλγεβρικό πιστοποιητικό μη-εφικτότητας (Farkas)

Η τελευταία γραμμή του τελικού tableau δεν είναι απλώς ένδειξη: κωδικοποιεί *απόδειξη*. Οι συντελεστές της γραμμής A_3 στις στήλες A_1, A_2, A_3 είναι $(-1, -1, 1)$, δηλαδή η γραμμή αυτή είναι ο γραμμικός συνδυασμός $-E_1 - E_2 + (-E_3) = -(E_1 + E_2 + E_3)$ των αρχικών εξισώσεων. Ισοδύναμα, προσθέτοντας απλώς τις τρεις αρχικές εξισώσεις:

$$E_1 + E_2 + E_3 : (1 - 1 + 1)x_1 + (1 + 2 - 3)x_2 + (1 + 1 - 2)x_3 + (1 - 3 + 2)x_4 + (1 + 1 - 2)x_5 = 2 + 1 - 4$$

$$\implies \boxed{x_1 = -1}$$

Όμως ο περιορισμός μη αρνητικότητας απαιτεί $x_1 \geq 0$. Άρα κανένα $x \geq 0$ δεν μπορεί να ικανοποιεί ταυτόχρονα τις τρεις εξισώσεις — το σύστημα είναι αποδεδειγμένα μη εφικτό, ανεξάρτητα από τον αλγόριθμο.

Παρατηρούμε επίσης ότι το σύστημα *χωρίς* τον περιορισμό $x \geq 0$ είναι συμβατό (έχει λύσεις με $x_1 = -1$): η μη εφικτότητα οφείλεται αποκλειστικά στη μη αρνητικότητα.

Α' Παράρτημα: πηγαίος κώδικας

Όλος ο κώδικας είναι σε Python 3 και χρησιμοποιεί numpy, scipy και matplotlib. Όπου απαιτείται εντοπισμός εκφυλισμού, οι υπολογισμοί γίνονται με *ακριβή ρητή αριθμητική* (fractions.Fraction) ώστε να μην υπάρχει κίνδυνος σφάλματος στρογγυλοποίησης. Κάθε αρχείο εκτελείται αυτόνομα και παράγει ακριβώς τα αποτελέσματα που παρουσιάζονται παραπάνω.

Α'.1 ex_1.py — γραφική επίλυση με αποσύνθεση

```

1  """Εργασία
2  1 - Άσκηση 1Γραφική
3  επίλυση του ΠΓΠ
4
5   $max\ Z = 3x_1 + 7x_2 + 4x_3 - 3x_4$ 
6  (P1)  $x_1 + 2x_2 \leq 6$       (P2)  $x_3 + x_4 \leq 6$ 
7  (P3)  $4x_1 + 5x_2 \leq 20$    (P4)  $-2x_1 + x_2 \leq 1$ 
8  (P5)  $2x_3 - x_4 \leq 4$      (P6)  $x_3 \geq 2$ 
9  (P7)  $1 \leq x_2 \leq 2$       (P8)  $x_1, x_4 \geq 0$ Κανένας
10
11  περιορισμός δεν αναμειγνύει τα  $(x_1, x_2)$  με τα  $(x_3, x_4)$ , άρα το πρόβλημα
12  αποσυντίθεται σε δύο ανεξάρτητα διάστατα2- ΠΓΠ, καθένα από τα οποία
13  λύνεται γραφικά. Ο κώδικας:
14  1. απαριθμεί ΟΛΕΣ τις τομές ζευγών ευθειών κάθε υποπροβλήματος,
15  2. κρατά όσες είναι εφικτές (= κορυφές του πολυγώνου),
16  3. μετράει πόσες ευθείες περνούν από κάθε κορυφή εντοπισμός( εκφυλισμού),
17  4. σχεδιάζει την εφικτή περιοχή, τις κορυφές και το διάνυσμα κλίσης,
18  5. επαληθεύει το συνολικό αποτέλεσμα με τη scipy.optimize.linprog.
19  """
20  import itertools
21  from fractions import Fraction as F
22
23  import matplotlib
24  matplotlib.use("Agg")
25  import matplotlib.pyplot as plt
26  import numpy as np
27  from scipy.optimize import linprog
28
29  TOL = F(0)
30
31
32  # ----- εργαλεία
33  def intersect(l1, l2):
34      """Τομή δύο ευθειών  $a*u + b*v = c$  ακριβής( αριθμητική με κλάσματα)."""
35      (a1, b1, c1), (a2, b2, c2) = l1[:3], l2[:3]
36      det = a1 * b2 - a2 * b1
37      if det == 0: # παράλληλες -> δεν ορίζεται κορυφή
38          return None
39      u = (c1 * b2 - c2 * b1) / det
40      v = (a1 * c2 - a2 * c1) / det
41      return (u, v)
42
43
44  def analyse(lines, obj, title, fname, xlabel, ylabel, xlim, ylim):
45      """Πλήρης γραφική ανάλυση ενός διάστατου2- ΠΓΠ.
46
47      lines: λίστα (a, b, c, sense, name) που εκφράζει  $a*u + b*v (<|=|>=) c$ 
48      obj : (c1, c2) συντελεστές της  $max\ c1*u + c2*v$ 
49      """
50      def feasible(p):
51          u, v = p
52          for a, b, c, sense, _ in lines:
53              lhs = a * u + b * v

```

```

54         if sense == "<=" and lhs > c + TOL:
55             return False
56         if sense == ">=" and lhs < c - TOL:
57             return False
58     return True
59
60     # ---- 1. όλες οι τομές ζευγών ευθειών ----
61     cand = {}
62     for l1, l2 in itertools.combinations(lines, 2):
63         p = intersect(l1, l2)
64         if p is None:
65             continue
66         cand.setdefault(p, set()).update([l1[4], l2[4]])
67
68     # ---- 2. ποιες είναι κορυφές του πολυγώνου ----
69     verts = {p: names for p, names in cand.items() if feasible(p)}
70
71     # ---- 3. ενεργοί περιορισμοί ανά κορυφή έλεγχος( εκφυλισμού) ----
72     print(f"\n===== {title} =====")
73     rows = []
74     for p in sorted(verts, key=lambda q: (float(q[0]), float(q[1]))):
75         u, v = p
76         active = [nm for a, b, c, s, nm in lines if a * u + b * v == c]
77         z = obj[0] * u + obj[1] * v
78         rows.append((p, active, z))
79         flag = " <-- ΕΚΦΥΛΙΣΜΕΝΗ (>2 ευθείες)" if len(active) > 2 else ""
80         print(f" ({str(u):>6}, {str(v):>6}) Z={str(z):>8} ενεργοί: "
81               f"{len(active)} {active}{flag}")
82     best = max(rows, key=lambda r: r[2])
83     print(f" ΒΕΛΤΙΣΤΗ ΚΟΡΥΦΗ: {tuple(str(t) for t in best[0])} Z*={best[2]}")
84
85     # ---- 4. σχεδίαση ----
86     plot_region(lines, rows, best, obj, title, fname, xlabel, ylabel, xlim, ylim)
87     return best
88
89
90 # ----- σχεδίαση -----
91 PALETTE = ["#1f77b4", "#d62728", "#2ca02c", "#9467bd",
92            "#8c564b", "#17becf", "#e377c2", "#7f7f7f"]
93
94
95 def plot_region(lines, rows, best, obj, title, fname, xlabel, ylabel, xlim, ylim):
96     """Σχεδίαση εφικτής περιοχής με ετικέτες που δεν αλληλεπικαλύπτονται.
97
98     Οι ετικέτες των κορυφών τοποθετούνται ΑΚΤΙΝΙΚΑ προς τα έξω μακριά από το
99     κέντρο βάρους του πολυγώνου), ώστε να μην πέφτουν η μία πάνω στην άλλη ούτε
100     μέσα στην εφικτή περιοχή. Η λεζάντα μπαίνει ΚΑΤΩ από τους άξονες.
101     """
102     fig, ax = plt.subplots(figsize=(7.4, 6.0))
103
104     # εφικτή περιοχή σκίαση()
105     gx = np.linspace(xlim[0], xlim[1], 800)
106     gy = np.linspace(ylim[0], ylim[1], 800)
107     UU, VV = np.meshgrid(gx, gy)
108     ok = np.ones_like(UU, dtype=bool)
109     for a, b, c, s, _ in lines:
110         lhs = float(a) * UU + float(b) * VV
111         ok &= (lhs <= float(c) + 1e-9) if s == "<=" else (lhs >= float(c) - 1e-9)
112     ax.contourf(UU, VV, ok.astype(float), levels=[0.5, 1.5],
113                colors=["#9fd3e8"], alpha=0.60, zorder=0)
114
115     # ευθείες περιορισμών - ρητά διαφορετικά χρώματα
116     for k, (a, b, c, s, nm) in enumerate(lines):
117         col = PALETTE[k % len(PALETTE)]

```

```

118     a, b, c = float(a), float(b), float(c)
119     if b != 0:
120         ax.plot(gx, (c - a * gx) / b, lw=1.4, color=col, label=nm, zorder=2)
121     else:
122         ax.axvline(c / a, lw=1.4, color=col, label=nm, zorder=2)
123
124     pts = np.array([[float(p[0]), float(p[1])] for p, _, _ in rows])
125     ax.plot(pts[:, 0], pts[:, 1], "o", color="crimson", ms=7, zorder=5)
126
127     cx, cy = pts[:, 0].mean(), pts[:, 1].mean()      # κέντρο βάρους κορυφών
128     sx = (xlim[1] - xlim[0]) or 1.0
129     sy = (ylim[1] - ylim[0]) or 1.0
130     bx, by = float(best[0][0]), float(best[0][1])
131     gvec = np.array([float(obj[0]) / sx, float(obj[1]) / sy])
132     gvec = gvec / (np.hypot(*gvec) or 1.0)
133
134     for (p, active, z) in rows:
135         px, py = float(p[0]), float(p[1])
136         d = np.array([(px - cx) / sx, (py - cy) / sy])
137         if np.hypot(*d) < 1e-9:
138             d = np.array([1.0, 1.0])
139         d = d / np.hypot(*d)
140         # η ετικέτα της βέλτιστης κορυφής πάει αντίθετα από το διάνυσμα κλίσης,
141         # ώστε να μην την καλύπτει το βέλος
142         if abs(px - bx) < 1e-9 and abs(py - by) < 1e-9:
143             d = -gvec
144         # και... αν η κορυφή είναι κολλητά στο περιθώριο, η ετικέτα γυρίζει
145         # προς τα μέσα ώστε να μη βγει έξω από τους άξονες
146         if px < xlim[0] + 0.12 * sx:
147             d[0] = abs(d[0])
148         elif px > xlim[1] - 0.12 * sx:
149             d[0] = -abs(d[0])
150         if py < ylim[0] + 0.12 * sy:
151             d[1] = abs(d[1])
152         elif py > ylim[1] - 0.12 * sy:
153             d[1] = -abs(d[1])
154         off = 17.0
155         ax.annotate(
156             f"({px:.2f}, {py:.2f}){'*' if len(active) > 2 else ''}\nZ={float(z):.2f}",
157             (px, py), textcoords="offset points",
158             xytext=(d[0] * off, d[1] * off), fontsize=8, zorder=8,
159             ha="left" if d[0] >= 0 else "right",
160             va="bottom" if d[1] >= 0 else "top",
161             bbox=dict(boxstyle="round,pad=0.22", fc="white", ec="0.75",
162                     alpha=0.88, lw=0.5))
163
164     # βέλτιστη κορυφή + διάνυσμα κλίσης
165     ax.plot([bx], [by], "o", color="gold", mec="k", ms=13, zorder=6)
166     L = 0.13
167     ax.annotate("", xy=(bx + gvec[0] * L * sx, by + gvec[1] * L * sy), xytext=(bx, by),
168                 arrowprops=dict(arrowstyle=">", lw=2.6, color="darkorange",
169                               mutation_scale=18), zorder=7)
170     ax.annotate(r"$\nabla Z$", (bx + gvec[0] * L * sx, by + gvec[1] * L * sy),
171                textcoords="offset points", xytext=(4 * np.sign(gvec[0] or 1), 4),
172                fontsize=9, color="darkorange", fontweight="bold", zorder=8,
173                ha="left" if gvec[0] >= 0 else "right")
174
175     ax.set_xlim(*xlim); ax.set_ylim(*ylim)
176     ax.set_xlabel(xlabel); ax.set_ylabel(ylabel)
177     ax.set_title(title, pad=10)
178     ax.grid(alpha=0.28, zorder=1)
179     ax.legend(fontsize=8, loc="upper center", bbox_to_anchor=(0.5, -0.11),
180              ncol=3, frameon=False)
181     fig.tight_layout(); fig.savefig(fname, dpi=170, bbox_inches="tight")

```

```

182 plt.close(fig)
183
184
185 # ----- Υποπρόβλημα 1: (x1, x2)
186 L1 = [
187     (F(1), F(2), F(6), "<=", "P1: x1+2x2<=6"),
188     (F(4), F(5), F(20), "<=", "P3: 4x1+5x2<=20"),
189     (F(-2), F(1), F(1), "<=", "P4: -2x1+x2<=1"),
190     (F(0), F(1), F(2), "<=", "P7a: x2<=2"),
191     (F(0), F(1), F(1), ">=", "P7b: x2>=1"),
192     (F(1), F(0), F(0), ">=", "P8a: x1>=0"),
193 ]
194 b1 = analyse(L1, (F(3), F(7)), "Υποπρόβλημα 1: max Z1 = 3x1 + 7x2",
195     "ex1_subproblem1.png", "x1", "x2", (-0.6, 4.6), (0.0, 2.8))
196
197 # ----- Υποπρόβλημα 2: (x3, x4)
198 L2 = [
199     (F(1), F(1), F(6), "<=", "P2: x3+x4<=6"),
200     (F(2), F(-1), F(4), "<=", "P5: 2x3-x4<=4"),
201     (F(1), F(0), F(2), ">=", "P6: x3>=2"),
202     (F(0), F(1), F(0), ">=", "P8b: x4>=0"),
203 ]
204 b2 = analyse(L2, (F(4), F(-3)), "Υποπρόβλημα 2: max Z2 = 4x3 - 3x4",
205     "ex1_subproblem2.png", "x3", "x4", (1.4, 4.4), (-0.6, 5.0))
206
207 # ----- Σύνθεση & επαλήθευση
208 x1s, x2s = b1[0]; x3s, x4s = b2[0]
209 Z = 3 * x1s + 7 * x2s + 4 * x3s - 3 * x4s
210 print("\n===== Συνολική λύση =====")
211 print(f"x* = ({x1s}, {x2s}, {x3s}, {x4s}), Z* = {Z}")
212
213 res = linprog([-3, -7, -4, 3],
214     A_ub=[[1, 2, 0, 0], [0, 0, 1, 1], [4, 5, 0, 0],
215           [-2, 1, 0, 0], [0, 0, 2, -1]],
216     b_ub=[6, 6, 20, 1, 4],
217     bounds=[(0, None), (1, 2), (2, None), (0, None)])
218 print(f"Επαλήθευση με scipy.linprog: x = {np.round(res.x, 6)}, Z = {-res.fun:.6f}")

```

Α.2 ex_2.py — χαρτοφυλάκιο, πλεονάζοντες περιορισμοί, σκιώδεις τιμές

```

1 """Εργασία
2 1 - Άσκηση 2Μοντελοποίηση
3 και γραφική επίλυση του προβλήματος βέλτιστου χαρτοφυλακίου.
4
5 max Z = 0.11 xD + 0.17 xF
6 (P1) xD + xF <= 12      συνολικό( κεφάλαιο)
7 (P2) xD <= 10           όριο( εγχώριων)
8 (P3) xF <= 7            όριο( ξένων)
9 (P4) -0.5 xD + xF >= 0   ξένες( >= μισό των εγχώριων)
10 (P5) xD - 0.5 xF >= 0    εγχώριες( >= μισό των ξένων)
11 xD, xF >= 00
12
13 κώδικας απαριθμεί όλες τις τομές ζευγών ευθειών, ξεχωρίζει τις κορυφές, ελέγχει
14 ποιοι περιορισμοί είναι πλεονάζοντες (redundant), σχεδιάζει την επιβλεπτική
15 περιοχή και επαληθεύει με scipy.linprog. Επιπλέον υπολογίζει τις σκιώδεις
16 τιμές (dual values) των δεσμευτικών περιορισμών.
17 """
18 import itertools
19 from fractions import Fraction as F
20
21 import matplotlib
22 matplotlib.use("Agg")

```

```

23 import matplotlib.pyplot as plt
24 import numpy as np
25 from scipy.optimize import linprog
26
27 # a*xD + b*xF (sense) c
28 LINES = [
29     (F(1), F(1), F(12), "<=", "P1: xD+xF<=12"),
30     (F(1), F(0), F(10), "<=", "P2: xD<=10"),
31     (F(0), F(1), F(7), "<=", "P3: xF<=7"),
32     (F(-1, 2), F(1), F(0), ">=", "P4: xF>=0.5xD"),
33     (F(1), F(-1, 2), F(0), ">=", "P5: xD>=0.5xF"),
34 ]
35 C = (F(11, 100), F(17, 100))
36
37
38 def feasible(p):
39     u, v = p
40     if u < 0 or v < 0:
41         return False
42     for a, b, c, s, _ in LINES:
43         lhs = a * u + b * v
44         if (s == "<=" and lhs > c) or (s == ">=" and lhs < c):
45             return False
46     return True
47
48
49 # ---- 1. όλες οι τομές ζευγών ευθειών -----
50 cand = {}
51 for (a1, b1, c1, _, n1), (a2, b2, c2, _, n2) in itertools.combinations(LINES, 2):
52     det = a1 * b2 - a2 * b1
53     if det == 0:
54         continue
55     p = ((c1 * b2 - c2 * b1) / det, (a1 * c2 - a2 * c1) / det)
56     cand.setdefault(p, set()).update([n1, n2])
57
58 verts = sorted([p for p in cand if feasible(p)], key=lambda q: float(C[0]*q[0]+C[1]*q[1]))
59 print("Κορυφές της εφικτής περιοχής:")
60 for p in verts:
61     act = [nm for a, b, c, s, nm in LINES if a*p[0] + b*p[1] == c]
62     z = C[0]*p[0] + C[1]*p[1]
63     print(f" (xD,xF)={str(p[0]):>5},{str(p[1]):>5} Z={float(z):8.4f} εκατ."
64           f" ενεργοί: {act}")
65 best = verts[-1]
66 print(f"\nΒΕΛΤΙΣΤΗn: xD*={best[0]}, xF*={best[1]}, Z*={float(C[0]*best[0]+C[1]*best[1]):.4f} εκατ.")
67
68 # ---- 2. έλεγχος πλεοναζόντων (redundant) περιορισμών -----
69 # Ένας περιορισμός είναι πλεονάζων όταν ΔΕΝ μπορεί να παραβιαστεί από κανένα
70 # σημείο που ικανοποιεί όλους τους υπόλοιπους: βελτιστοποιούμε το αριστερό
71 # του μέλος πάνω στην περιοχή των υπολοίπων και συγκρίνουμε με το b.
72 print("\nΈλεγχοςn πλεοναζόντων (redundant) περιορισμών:")
73 for k, (a, b, c, s, nm) in enumerate(LINES):
74     rest = LINES[k:] + LINES[k+1:]
75     A_ub, b_ub = [], []
76     for aa, bb, cc, ss, _ in rest:
77         A_ub.append([float(aa), float(bb)] if ss == "<=" else [-float(aa), -float(bb)])
78         b_ub.append(float(cc) if ss == "<=" else -float(cc))
79     # <= : μεγιστοποιούμε το LHS · >= : ελαχιστοποιούμε το LHS
80     obj = [-float(a), -float(b)] if s == "<=" else [float(a), float(b)]
81     r = linprog(obj, A_ub=A_ub, b_ub=b_ub, bounds=[(0, None)]*2)
82     if r.status != 0:
83         # μη φραγμένο -> μπορεί να παραβιαστεί
84         print(f" {nm:<18}: ΟΧΙ πλεονάζων το ( LHS δεν είναι φραγμένο)")
85         continue
86     ext = -r.fun if s == "<=" else r.fun
87     red = (ext <= float(c) + 1e-9) if s == "<=" else (ext >= float(c) - 1e-9)

```

```

87     print(f" {nm:<18}: ακρότατο LHS στους υπόλοιπους = {ext:8.4f} vs b = {float(c):6.2f}"
88           f" -> ΠΛΕΟΝΑΖΩΝ{' ' if red else OXI} πλεονάζων'")
89
90 # ---- 3. επαλήθευση + σκιδώεις τιμές -----
91 res = linprog([-0.11, -0.17],
92              A_ub=[[1, 1], [1, 0], [0, 1], [0.5, -1], [-1, 0.5]],
93              b_ub=[12, 10, 7, 0, 0], bounds=[(0, None)]*2)
94 print(f"\nscipy.linprog: x={np.round(res.x,6)} Z={-res.fun:.4f}")
95
96 # Οι σκιδώεις τιμές προκύπτουν από τους ΔΕΣΜΕΥΤΙΚΟΥΣ περιορισμούς: αν B είναι
97 # ο πίνακας με γραμμές τα διανύσματα των δεσμευτικών περιορισμών, τότε το y
98 # λύνει το  $B^T y = c$  συνθήκη( βελτιστότητας στη βέλτιστη κορυφή).
99 binding = [(nm, [float(a), float(b)]) for a, b, c, s, nm in LINES
100            if a*best[0] + b*best[1] == c]
101 B = np.array([row for _, row in binding])
102 y = np.linalg.solve(B.T, np.array([0.11, 0.17]))
103 print("\nΣκιδώεις τιμές μόνο( οι δεσμευτικοί έχουν μη μηδενική τιμή):")
104 for (nm, _) , yi in zip(binding, y):
105     print(f" {nm:<18} y = {yi:.4f}")
106 for a, b, c, s, nm in LINES:
107     if a*best[0] + b*best[1] != c:
108         print(f" {nm:<18} y = 0.0000 μη( δεσμευτικός)")
109
110 # ---- 4. σχήμα -----
111 # Οι ετικέτες των κορυφών τοποθετούνται ΑΚΤΙΝΙΚΑ προς τα έξω μακριά( από το
112 # κέντρο βάρους του πολυγώνου) ώστε να μην αλληλεπικαλύπτονται, και η λεζάντα
113 # μπαίνει κάτω από τους άξονες.
114 PALETTE = ["#1f77b4", "#d62728", "#2ca02c", "#9467bd", "#8c564b"]
115 XL, YL = (0, 13), (0, 13)
116
117 fig, ax = plt.subplots(figsize=(7.4, 6.0))
118 g = np.linspace(XL[0], XL[1], 800)
119 UU, VV = np.meshgrid(g, g)
120 ok = (UU >= 0) & (VV >= 0)
121 for a, b, c, s, _ in LINES:
122     lhs = float(a)*UU + float(b)*VV
123     ok &= (lhs <= float(c) + 1e-9) if s == "<=" else (lhs >= float(c) - 1e-9)
124 ax.contourf(UU, VV, ok.astype(float), levels=[.5, 1.5], colors=["#9fd3e8"],
125             alpha=.60, zorder=0)
126
127 for k, (a, b, c, s, nm) in enumerate(LINES):
128     col = PALETTE[k % len(PALETTE)]
129     a, b, c = float(a), float(b), float(c)
130     if b != 0:
131         ax.plot(g, (c - a*g)/b, lw=1.4, color=col, label=nm, zorder=2)
132     else:
133         ax.axvline(c/a, lw=1.4, color=col, label=nm, zorder=2)
134
135 P = np.array([[float(p[0]), float(p[1])] for p in verts])
136 ax.plot(P[:, 0], P[:, 1], "o", color="crimson", ms=7, zorder=5)
137
138 cx, cy = P[:, 0].mean(), P[:, 1].mean()
139 bx, by = float(best[0]), float(best[1])
140 gvec = np.array([0.11, 0.17]); gvec = gvec / np.hypot(*gvec)
141
142 for p in verts:
143     px, py = float(p[0]), float(p[1])
144     d = np.array([px - cx, py - cy])
145     if np.hypot(*d) < 1e-9:
146         d = np.array([-1.0, -1.0])
147     d = d / np.hypot(*d)
148     if abs(px - bx) < 1e-9 and abs(py - by) < 1e-9:
149         d = -gvec # η ετικέτα αντίθετα από το βέλος
150     # κορυφή κολλητά στο περιθώριο -> η ετικέτα γυρίζει προς τα μέσα

```

```

151     if px < XL[0] + 0.12 * (XL[1] - XL[0]):
152         d[0] = abs(d[0])
153     elif px > XL[1] - 0.12 * (XL[1] - XL[0]):
154         d[0] = -abs(d[0])
155     if py < YL[0] + 0.12 * (YL[1] - YL[0]):
156         d[1] = abs(d[1])
157     elif py > YL[1] - 0.12 * (YL[1] - YL[0]):
158         d[1] = -abs(d[1])
159     ax.annotate(f"({px:.2f}, {py:.2f})\nZ={float(C[0]*p[0]+C[1]*p[1]):.3f}",
160                (px, py), textcoords="offset points",
161                xytext=(d[0]*20, d[1]*20), fontsize=8, zorder=8,
162                ha="left" if d[0] >= 0 else "right",
163                va="bottom" if d[1] >= 0 else "top",
164                bbox=dict(boxstyle="round,pad=0.22", fc="white", ec="0.75",
165                        alpha=0.88, lw=0.5))
166
167 ax.plot([bx], [by], "o", color="gold", mec="k", ms=13, zorder=6)
168 L = 1.9
169 ax.annotate("", xy=(bx + gvec[0]*L, by + gvec[1]*L), xytext=(bx, by),
170            arrowprops=dict(arrowstyle="->", lw=2.6, color="darkorange",
171                            mutation_scale=18), zorder=7)
172 ax.annotate(r"$\nabla Z$", (bx + gvec[0]*L, by + gvec[1]*L),
173            textcoords="offset points", xytext=(5, 3), fontsize=9,
174            color="darkorange", fontweight="bold", zorder=8)
175
176 ax.set_xlim(*XL); ax.set_ylim(*YL)
177 ax.set_xlabel("xD εκατ(. € σε εγχώριες)"); ax.set_ylabel("xF εκατ(. € σε ξένες)")
178 ax.set_title("Βέλτιστο χαρτοφυλάκιο: max Z = 0.11 xD + 0.17 xF", pad=10)
179 ax.grid(alpha=.28, zorder=1)
180 ax.legend(fontsize=8, loc="upper center", bbox_to_anchor=(0.5, -0.11),
181          ncol=3, frameon=False)
182 fig.tight_layout(); fig.savefig("ex2_graph.png", dpi=170, bbox_inches="tight")

```

Α.3 ex_3.py — δύο μετασχηματισμοί και σύνολο βέλτιστων λύσεων

```

1  """Εργασία
2  1 - Άσκηση 3Μετατροπή
3  του προβλήματος
4
5      min Z = |x1| + |x2| + |x3|
6      s.t.  x1 - 2x2 = 3 ,   -x2 + x3 <= 1 ,   x1, x2, x3 ελεύθερεςσε
7
8  ισοδύναμο ΓΠ, με ΔΥΟ διαφορετικούς ισοδυναμους() μετασχηματισμούς:
9
10  A() Διάσπαση μεταβλητών:  xj = xj+ - xj- ,   |xj| = xj+ + xj- ,   xj± >= 0
11  B() Άνω φράγματα:         tj >= xj , tj >= -xj δηλ(. tj >= |xj|) , min Σ tj0
12
13  κώδικας λύνει και τα δύο, επαληθεύει ότι δίνουν την ίδια βέλτιστη τιμή, και
14  - επιπλέον - απαριθμεί ΟΛΟ το σύνολο των βέλτιστων λύσεων το( πρόβλημαέχει
15  άπειρες βέλτιστες λύσεις: μια ολόκληρη ακμή).
16  """
17  import numpy as np
18  from scipy.optimize import linprog
19
20  print("=== A() Διάσπαση μεταβλητών: μεταβλητές [x1+,x1-,x2+,x2-,x3+,x3-] ===")
21  A_eq = [[1, -1, -2, 2, 0, 0]]          # x1 - 2x2 = 3
22  b_eq = [3]
23  A_ub = [[0, 0, -1, 1, 1, -1]]          # -x2 + x3 <= 1
24  b_ub = [1]
25  rA = linprog([1]*6, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
26              bounds=[(0, None)]*6)
27  xp = rA.x

```

```

28 x = [xp[0]-xp[1], xp[2]-xp[3], xp[4]-xp[5]]
29 print(f" λύση (split): {np.round(xp,6)}")
30 print(f" x = {np.round(x,6)}, Z* = {rA.fun:.6f}")
31 print(f" έλεγχος συμπληρωματικότητας  $x_j + x_{j-} =$  "
32       f"{[round(xp[2*i]*xp[2*i+1],9) for i in range(3)]}")
33
34 print("\n=== B() Μετασχηματισμός με άνω φράγματα: [x1,x2,x3,t1,t2,t3] ===")
35 #  $t_j - x_j \geq 0 \rightarrow -t_j + x_j \leq 0$  ;  $t_j + x_j \geq 0 \rightarrow -t_j - x_j \leq 0$ 
36 A_ub2 = [[1, 0, 0, -1, 0, 0], [-1, 0, 0, -1, 0, 0],
37          [0, 1, 0, 0, -1, 0], [0, -1, 0, 0, -1, 0],
38          [0, 0, 1, 0, 0, -1], [0, 0, -1, 0, 0, -1],
39          [0, -1, 1, 0, 0, 0]]
40 b_ub2 = [0, 0, 0, 0, 0, 1]
41 rB = linprog([0, 0, 0, 1, 1, 1], A_ub=A_ub2, b_ub=b_ub2,
42             A_eq=[[1, -2, 0, 0, 0, 0]], b_eq=[3],
43             bounds=[(None, None)]*3 + [(0, None)]*3)
44 print(f" x = {np.round(rB.x[:3],6)}, t = {np.round(rB.x[3:],6)}, Z* = {rB.fun:.6f}")
45
46 print("\n=== Το ΣΥΝΟΛΟ των βέλτιστων λύσεων ===")
47 # Παραμετροποίηση:  $x_1 = 3 + 2*x_2$  από τον ο1 περιορισμό.
48 # Το βέλτιστο  $x_3$  δεδομένου  $x_2$  είναι  $x_3 = \min(0, 1+x_2)$  ελαχιστοποιεί( το  $|x_3|$ ).
49 for x2 in np.linspace(-2.0, 0.5, 26):
50     x1 = 3 + 2*x2
51     x3 = min(0.0, 1 + x2)
52     print(f" x2={x2:6.2f} -> x={x1:6.2f},{x2:6.2f},{x3:6.2f} Z={abs(x1)+abs(x2)+abs(x3):7.3f}"
53           + (" <-- ΒΕΛΤΙΣΤΟ" if abs(abs(x1)+abs(x2)+abs(x3) - 2) < 1e-9 else ""))
54 print("\n Το βέλτιστο επιτυγχάνεται για κάθε x2 στο [-3/2, -1]:")
55 print(" x2 = -1 -> x = ( 1, -1, 0) Z = 2 κορυφή()")
56 print(" x2 = -3/2 -> x = ( 0, -3/2,-1/2) Z = 2 κορυφή()")
57 print(" δηλ. το σύνολο των βέλτιστων λύσεων είναι η ακμή που ενώνει τις δύο κορυφές.")

```

Α'.4 ex_4.py — αριθμητική επαλήθευση των μοντέλων MMM

```

1 """Εργασία
2 1 - Άσκηση 4Το
3 ζητούμενο είναι μοντελοποίηση· ο κώδικας εδώ υπάρχει ΜΟΝΟ για να επιβεβαιώσει
4 ότι τα δύο μοντέλα α() και β() είναι σωστά διατυπωμένα και δίνουν
5 το αναμενόμενο αποτέλεσμα σε ένα μικρό αριθμητικό στιγμιότυπο.α
6
7 () max  $\sum_k g_k d_k - \sum_k \sum_r c_{rk} x_{rk}$ 
8 s.t.  $\sum_r f_{rk} x_{rk} \geq d_k$  πλήρης( κάλυψη ζήτησης)
9  $\sum_k x_{rk} \leq v_r$  ,  $x_{rk}$  ακέραια  $\geq 0$ 
10
11 () max  $\sum_k g_k y_k - \sum_k \sum_r c_{rk} x_{rk}$ 
12 s.t.  $0.95 d_k \leq \sum_r f_{rk} x_{rk} \leq 1.05 d_k$ 
13  $y_k \leq d_k$  ,  $y_k \leq \sum_r f_{rk} x_{rk}$ 
14  $\sum_k x_{rk} \leq v_r$  ,  $x_{rk}$  ακέραια  $\geq 0$  ,  $y_k \geq 0$  Χρησιμοποιείται
15
16 η PuLP με τον επιλυτή CBC, όπως στις διαλέξεις.
17 """
18 import pulp
19
20 # ----- στιγμιότυπο δοκιμής
21 K, R = 3, 2 # 3 διαδρομές, 2 τύποι οχημάτων
22 ROUTES = range(1, K + 1)
23 TYPES = range(1, R + 1)
24
25 d = {1: 500, 2: 300, 3: 420} # ζήτηση ανά διαδρομή
26 g = {1: 25.0, 2: 30.0, 3: 20.0} # έσοδο ανά επιβάτη
27 f = {(1, 1): 120, (1, 2): 100, (1, 3): 110, # χωρητικότητα τύπου r στη k
28      (2, 1): 80, (2, 2): 90, (2, 3): 75}
29 c = {(1, 1): 900., (1, 2): 850., (1, 3): 880., # κόστος τύπου r στη k

```

```

30     (2, 1): 600., (2, 2): 640., (2, 3): 610.}
31 v = {1: 6, 2: 8} # διαθέσιμα οχήματα ανά τύπο
32
33 solver = pulp.PULP_CBC_CMD(msg=False)
34
35
36 # =====
37 # Μοντέλο α(): η ζήτηση εξυπηρετείται στο ακέραιο
38 # =====
39 def model_a():
40     m = pulp.LpProblem('MMM_plhrhs_kalypsis', pulp.LpMaximize)
41
42     # Μεταβλητές απόφασης: ακέραιος αριθμός οχημάτων τύπου r στη διαδρομή k
43     x = pulp.LpVariable.dicts('x', (TYPES, ROUTES), lowBound=0, cat='Integer')
44
45     # Τα έσοδα Σ g_k d_k είναι ΣΤΑΘΕΡΑ όλη( η ζήτηση εξυπηρετείται), οπότε η
46     # μεγιστοποίηση του κέρδους ισοδυναμεί με ελαχιστοποίηση του κόστους.
47     esoda = sum(g[k] * d[k] for k in ROUTES)
48     m += esoda - pulp.lpSum(c[r, k] * x[r][k] for r in TYPES for k in ROUTES)
49
50     for k in ROUTES: # πλήρης κάλυψη της ζήτησης
51         m += pulp.lpSum(f[r, k] * x[r][k] for r in TYPES) >= d[k], f'zhthsh_{k}'
52     for r in TYPES: # διαθέσιμος στόλος
53         m += pulp.lpSum(x[r][k] for k in ROUTES) <= v[r], f'stolos_{r}'
54
55     m.solve(solver)
56     return m, x, esoda
57
58
59 # =====
60 # Μοντέλο β(): χωρητικότητα στο [95%, 105%] της ζήτησης
61 # =====
62 def model_b():
63     m = pulp.LpProblem('MMM_euelikth_kalypsi', pulp.LpMaximize)
64
65     x = pulp.LpVariable.dicts('x', (TYPES, ROUTES), lowBound=0, cat='Integer')
66     y = pulp.LpVariable.dicts('y', ROUTES, lowBound=0) # επιβάτες που εξυπηρετούνται
67
68     m += (pulp.lpSum(g[k] * y[k] for k in ROUTES)
69           - pulp.lpSum(c[r, k] * x[r][k] for r in TYPES for k in ROUTES))
70
71     for k in ROUTES:
72         cap = pulp.lpSum(f[r, k] * x[r][k] for r in TYPES)
73         m += cap >= 0.95 * d[k], f'kato_orio_{k}' # >= 95% της ζήτησης
74         m += cap <= 1.05 * d[k], f'anw_orio_{k}' # <= 105% της ζήτησης
75         m += y[k] <= d[k], f'y_leq_d_{k}' # όχι πάνω από τη ζήτηση
76         m += y[k] <= cap, f'y_leq_cap_{k}' # όχι πάνω από τις θέσεις
77     for r in TYPES:
78         m += pulp.lpSum(x[r][k] for k in ROUTES) <= v[r], f'stolos_{r}'
79
80     m.solve(solver)
81     return m, x, y
82
83
84 # =====
85 ma, xa, esoda = model_a()
86 print('=== Μοντέλο α(): πλήρης κάλυψη της ζήτησης ===')
87 print(f' κατάσταση: {pulp.LpStatus[ma.status]}')
88 print(' ' + ''.join(f'διαδρ{"." + str(k) + ">12}" for k in ROUTES))
89 for r in TYPES:
90     print(f' τύπος {r}: ' + ''.join(f'{int(xa[r][k].value())>12}' for k in ROUTES))
91 cap_a = {k: sum(f[r, k] * xa[r][k].value() for r in TYPES) for k in ROUTES}
92 print(' χωρητικότ.' + ''.join(f'{int(cap_a[k])>12}' for k in ROUTES))
93 print(' ζήτηση ' + ''.join(f'{d[k]>12}' for k in ROUTES))

```

```

94     + f' (cap >= d παντού: {all(cap_a[k] >= d[k] for k in ROUTES)})')
95     kostos = esoda - pulp.value(ma.objective)
96     print(f' έσοδα = {esoda:.2f}, κόστος = {kostos:.2f}, '
97           f' ΚΕΡΔΟΣ = {pulp.value(ma.objective):.2f}')
98
99     mb, xb, yb = model_b()
100    print('\n=== Μοντέλο β(): χωρητικότητα στο [95%, 105%] της ζήτησης ===')
101    print(f' κατάσταση: {pulp.LpStatus[mb.status]}')
102    print(' ' + ''.join(f'διαδρ{"."+str(k):>12}' for k in ROUTES))
103    for r in TYPES:
104        print(f' τύπος {r}: ' + ''.join(f'int(xb[r][k].value()):>12}' for k in ROUTES))
105    cap_b = {k: sum(f[r, k] * xb[r][k].value() for r in TYPES) for k in ROUTES}
106    print(' χωρητικότη: ' + ''.join(f'int(cap_b[k]):>12}' for k in ROUTES))
107    print(' επιτρεπτό ' + ''.join(f'f"[{0.95*d[k]:.0f},{1.05*d[k]:.0f}]":>12}' for k in ROUTES))
108    print(' y_k ' + ''.join(f'int(yb[k].value()):>12}' for k in ROUTES))
109    print(' min(d,cap)' + ''.join(f'int(min(d[k], cap_b[k])):>12}' for k in ROUTES))
110    + ' <- ταυτίζονται, όπως αναμενόταν')
111    print(f' ΚΕΡΔΟΣ = {pulp.value(mb.objective):.2f}')
112
113    print('\nΠαρατήρηση: στο β() το y_k δέχεται αυτόματα στο min(d_k, χωρητικότητα)')
114    print('επειδή g_k > 0 και η αντικειμενική συνάρτηση το μεγιστοποιεί.')

```

Α.5 ex_5.py — απαρίθμηση τομών, βασικών λύσεων και κορυφών

```

1  """Εργασία
2  1 - Άσκηση 5
3      max Z = -2x1 + x2 - 4x3 + 3x4
4      (P1) x1 + x2 + 3x3 + 2x4 <= 4
5      (P2) x1 - x3 + x4 <= 2
6      (P3) 2x1 + x2 <= 3
7      x1..x4 >= 0
8
9  () Όλες οι τομές n=4 υπερεπιπέδων από τα m+n=7 -> C(7,4)=35 συνδυασμοί.
10     Ξεχωρίζουμε ποιες είναι κορυφές του πολυτόπου και ποιες εκφυλισμένες.β
11  () Με μεταβλητές χαλάρωσης s1,s2,s3 -> 7 μεταβλητές, 3 εξισώσεις.
12     Όλες οι βασικές λύσεις = C(7,3) = 35 επιλογές βάσης.γ
13  () Αντιστοίχιση βασικών λύσεων <-> κορυφών και βέλτιστη λύση.Όλοι
14
15     οι υπολογισμοί γίνονται με AKPIBH ρητή αριθμητική (fractions), ώστε να μην
16     υπάρχει καμία αμφιβολία για τον εντοπισμό των μηδενικών εκφυλισμών().
17
18  """
19  import itertools
20  from fractions import Fraction as F
21
22  A = [[1, 1, 3, 2], [1, 0, -1, 1], [2, 1, 0, 0]]
23  b = [4, 2, 3]
24  c = [-2, 1, -4, 3]
25  A = [F(v) for v in r] for r in A
26  b = [F(v) for v in b]
27  c = [F(v) for v in c]
28  m, n = 3, 4
29
30  def gauss(M, r):
31      """Λύση τετραγωνικού συστήματος με ακριβή αριθμητική. None αν ιδιάζον."""
32      k = len(r)
33      M = [row[:] for row in M]
34      r = r[:]
35      for i in range(k):
36          p = next((j for j in range(i, k) if M[j][i] != 0), None)
37          if p is None:
38              return None

```

```

39     M[i], M[p] = M[p], M[i]
40     r[i], r[p] = r[p], r[i]
41     pv = M[i][i]
42     M[i] = [v / pv for v in M[i]]
43     r[i] /= pv
44     for j in range(k):
45         if j != i and M[j][i] != 0:
46             f = M[j][i]
47             M[j] = [M[j][t] - f * M[i][t] for t in range(k)]
48             r[j] -= f * r[i]
49     return r
50
51
52 # =====
53 # α() Τομές υπερεπιπέδων στον  $R^4$ 
54 # =====
55 H = [(A[i][:], b[i], f"P{i+1}") for i in range(m)]
56 for j in range(n):
57     e = [F(0)] * n
58     e[j] = F(1)
59     H.append((e, F(0), f"x{j+1}=0"))
60
61 print("=" * 78)
62 print("α() ΤΟΜΕΣ 4 ΥΠΕΡΕΠΙΠΕΔΩΝ ΑΠΟ 7 -> C(7,4) = 35")
63 print("=" * 78)
64 sing = []
65 pts = {}
66 inf = 0
67 for comb in itertools.combinations(range(7), 4):
68     M = [H[i][0][:] for i in comb]
69     rhs = [H[i][1] for i in comb]
70     lab = " n ".join(H[i][2] for i in comb)
71     sol = gauss(M, rhs)
72     if sol is None:
73         sing.append(lab)
74         continue
75     feas = all(v >= 0 for v in sol) and \
76         all(sum(A[i][j] * sol[j] for j in range(n)) <= b[i] for i in range(m))
77     if feas:
78         pts.setdefault(tuple(sol), []).append(lab)
79     else:
80         inf += 1
81 print(f"ιδιάζοντα παράλληλα(, χωρίς μοναδική τομή): {len(sing)}")
82 print(f"τομές που ΔΕΝ είναι εφικτές : {inf}")
83 print(f"τομές που είναι ΕΦΙΚΤΕΣ : {sum(len(v) for v in pts.values())}")
84 print(f"ΔΙΑΚΡΙΤΕΣ κορυφές του πολυτόπου : {len(pts)}\n")
85 for p in sorted(pts, key=lambda q: [float(v) for v in q]):
86     act = [f"P{i+1}" for i in range(m)]
87     if sum(A[i][j] * p[j] for j in range(n)) == b[i]:
88         act += [f"x{j+1}=0" for j in range(n) if p[j] == 0]
89     z = sum(c[j] * p[j] for j in range(n))
90     tag = " <= ΕΚΦΥΛΙΣΜΕΝΗ" if len(act) > n else ""
91     print(f" x={{'.'.join(str(v) for v in p):<16)} Z={str(z):>6} "
92           f"ενεργά υπερεπίπεδα: {len(act):>2} {act}{tag}")
93
94 # =====
95 # β() Βασικές λύσεις του συστήματος με μεταβλητές χαλάρωσης
96 # =====
97 As = [A[i] + [F(1) if j == i else F(0) for j in range(m)] for i in range(m)]
98 cs = c + [F(0)] * m
99 nm = ["x1", "x2", "x3", "x4", "s1", "s2", "s3"]
100
101 print("\n" + "=" * 78)
102 print("β() ΟΛΕΣ ΟΙ ΒΑΣΙΚΕΣ ΛΥΣΕΙΣ -> C(7,3) = 35 επιλογές βάσης")

```

```

103 print("=" * 78)
104 print(f"{'#':>3} Βάση{'':<14} " + "...".join(f"{'v':>7}" for v in nm) + f"{'Z':>8} Χαρακτηρισμός")
105 bfs, deg, nb = 0, 0, 0
106 table = []
107 for k, comb in enumerate(itertools.combinations(range(7), 3), 1):
108     M = [[As[i][j] for j in comb] for i in range(m)]
109     sol = gauss(M, b)
110     Bn = "{" + "...".join(nm[j] for j in comb) + "}"
111     if sol is None:
112         nb += 1
113         print(f"{'k':>3} {'Bn':<14} " + " " * 49 + " ιδιάζων B δεν (είναι βάση)")
114         table.append((k, Bn, None, None, "ιδιάζων B"))
115         continue
116     x = [F(0)] * 7
117     for t, j in enumerate(comb):
118         x[j] = sol[t]
119     feas = all(v >= 0 for v in x)
120     d = feas and any(v == 0 for v in sol)
121     z = sum(cs[j] * x[j] for j in range(7))
122     st = ("ΒΕΦΛ" + (" ΕΚΦΥΛΙΣΜΕΝΗ" if d else "")) if feas else "μηεφικτή- βασική λύση"
123     bfs += feas
124     deg += d
125     print(f"{'k':>3} {'Bn':<14} " + "...".join(f"{'str(v)':>7}" for v in x) +
126           f"{'str(z):>8} {'st}"))
127     table.append((k, Bn, x, z, st))
128
129 print(f"\ΣΥΝΟΨΗ: 35 συνδυασμοί = {nb} ιδιάζοντες + {35-nb} βασικές λύσεις")
130 print(f"        από τις {35-nb} βασικές λύσεις οι {bfs} είναι ΕΦΙΚΤΕΣ ΒΕΦΛ()")
131 print(f"        από τις {bfs} ΒΕΦΛ οι {deg} είναι ΕΚΦΥΛΙΣΜΕΝΕΣ")
132
133 # =====
134 # γ() Αντιστοίχιση και βέλτιστη λύση
135 # =====
136 print("\n" + "=" * 78)
137 print("γ() ΑΝΤΙΣΤΟΙΧΙΣΗ ΒΕΦΛ <-> ΚΟΡΥΦΩΝ")
138 print("=" * 78)
139 groups = {}
140 for k, Bn, x, z, st in table:
141     if x is None or not all(v >= 0 for v in x):
142         continue
143     groups.setdefault(tuple(x[:4]), []).append(Bn)
144 for v, bs in sorted(groups.items(), key=lambda t: float(sum(c[j]*t[0][j] for j in range(4)))):
145     z = sum(c[j] * v[j] for j in range(4))
146     print(f" κορυφή ({', '.join(str(t) for t in v):<14}) Z={str(z):>6} <- {len(bs)} βάση/βάσεις/: {bs}")
147 best = max(groups, key=lambda v: sum(c[j] * v[j] for j in range(4)))
148 print(f"\ΒΕΛΤΙΣΤΗΝ ΚΟΡΥΦΗ: x* = {tuple(str(v) for v in best)}, "
149       f"Z* = {sum(c[j]*best[j] for j in range(4))}")
150 print(f"Είναι ΕΚΦΥΛΙΣΜΕΝΗ: της αντιστοιχούν {len(groups[best])} διαφορετικές βάσεις.")
151
152 # επαλήθευση
153 from scipy.optimize import linprog
154 import numpy as np
155 r = linprog([2, -1, 4, -3], A_ub=[[1, 1, 3, 2], [1, 0, -1, 1], [2, 1, 0, 0]],
156            b_ub=[4, 2, 3], bounds=[(0, None)] * 4)
157 print(f"\Επαλήθευση scipy.linprog: x={np.round(r.x,6)}, Z={-r.fun:.6f}")

```

Α.6 ex_6.py — Simplex και πλήρης κατασκευή του adjacency graph

```

1 """Εργασία
2 1 - Άσκηση 6
3     max Z = 3x1 + 5x2 + 6x3
4     2x1 + x2 + x3 <= 4 ; x1 + 2x2 + x3 <= 4

```

```

5      x1 + x2 + 2x3 <= 4 ; x1 + x2 + x3 <= 3 ; x >= 0α
6
7      () Simplex με τυπωμένα tableaux σε ακριβή ρητή αριθμητική.β
8      () ΠΛΗΡΗΣ διερεύνηση όλων των εναλλακτικών διαδρομών: σε κάθε κόμβο
9      δοκιμάζονται ΟΛΕΣ οι εισερχόμενες με θετικό ( $c_j - z_j$ ) και ΟΛΕΣ οι
10     ισοπαλίες του ratio test. Παράγεται ο Simplex adjacency graph κόμβοι( =
11     βάσεις/κορυφές/, ακμές = επιτρεπτά pivots) και εντοπίζονται οι
12     εκφυλισμένες κορυφές περισσότερες( από μία βάσεις στο ίδιο σημείο) και
13     τα εκφυλισμένα pivots αλλαγή( βάσης χωρίς μεταβολή του σημείου / του Z).
14     """
15     from fractions import Fraction as F
16
17     A = [[2, 1, 1], [1, 2, 1], [1, 1, 2], [1, 1, 1]]
18     b = [4, 4, 4, 3]
19     c = [3, 5, 6]
20     m, n = 4, 3
21     A = [[F(v) for v in r] for r in A]
22     b = [F(v) for v in b]
23     cost = [F(v) for v in c] + [F(0)] * m
24     M = [A[i] + [F(1) if j == i else F(0) for j in range(m)] for i in range(m)]
25     nm = ["x1", "x2", "x3", "s1", "s2", "s3", "s4"]
26
27
28     def build(basis):
29         """Πλήρες tableau για τη δοσμένη βάση (Gauss-Jordan, ακριβής αριθμητική)."""
30         T = [[M[i][j] for j in range(7)] + [b[i]] for i in range(m)]
31         for r, bi in enumerate(basis):
32             p = next((k for k in range(r, m) if T[k][bi] != 0), None)
33             if p is None:
34                 return None
35             T[r], T[p] = T[p], T[r]
36             pv = T[r][bi]
37             T[r] = [v / pv for v in T[r]]
38             for k in range(m):
39                 if k != r and T[k][bi] != 0:
40                     f = T[k][bi]
41                     T[k] = [T[k][t] - f * T[r][t] for t in range(8)]
42             # γραμμή κριτηρίου:  $c_j - z_j$  (max: βέλτιστο όταν όλα <= 0)
43             red = [cost[j] - sum(cost[basis[i]] * T[i][j] for i in range(m)) for j in range(7)]
44             z = sum(cost[basis[i]] * T[i][7] for i in range(m))
45             return T, red, z
46
47
48     def point(basis, T):
49         x = [F(0)] * 7
50         for i, bi in enumerate(basis):
51             x[bi] = T[i][7]
52         return tuple(x[:3])
53
54
55     def show(tag, basis, T, red, z):
56         print(f"\n--- {tag} Βάση = ({', '.join(nm[j] for j in basis)}), Z = {z}")
57         print(" " + "".join(f"{v:>8}" for v in nm) + f"{'RHS':>8}")
58         for i in range(m):
59             print(f" {nm[basis[i]]:<6}" + "".join(f"{str(T[i][j]):>8}" for j in range(7))
60                   + f"{str(T[i][7]):>8}")
61         print(f" c-z " + "".join(f"{str(red[j]):>8}" for j in range(7)) + f"{str(z):>8}")
62
63
64     # =====
65     print("=" * 78)
66     print("α() ΕΚΤΕΛΕΣΗ SIMPLEX κανόνας( Dantzig: μεγαλύτερο  $c_j - z_j$ ")
67     print("=" * 78)
68     basis = [3, 4, 5, 6]

```

```

69 it = 0
70 while True:
71     T, red, z = build(basis)
72     show(f"Επανάληψη {it}", basis, T, red, z)
73     cand = [j for j in range(7) if j not in basis and red[j] > 0]
74     if not cand:
75         print("\n Όλα τα c_j - z_j <= 0 => ΒΕΛΤΙΣΤΟ")
76         print(f" x* = {tuple(str(v) for v in point(basis, T))}, Z* = {z}")
77         break
78     e = max(cand, key=lambda j: red[j])
79     rat = [(T[i][7] / T[i][e], i) for i in range(m) if T[i][e] > 0]
80     mn = min(r for r, _ in rat)
81     ties = [i for r, i in rat if r == mn]
82     print(f" εισερχόμενη: {nm[e]} (c-z = {red[e]})")
83     print(" ratio test: " + ", ".join(f"{nm[basis[i]]}: {T[i][7]}/{T[i][e]} = {T[i][7]/T[i][e]}"
84                                     for _, i in rat)
85           + f" -> εξερχόμενη: {nm[basis[ties[0]]}]"
86           + (" ΙΣΟΠΑΛΙΑ (-> εκφυλισμός στην επόμενη κορυφή)" if len(ties) > 1 else ""))
87     basis = basis[:ties[0]] + [e] + basis[ties[0] + 1:]
88     it += 1
89
90 # =====
91 print("\n" + "=" * 78)
92 print("β() SIMPLEX ADJACENCY GRAPH - ΟΛΕΣ ΟΙ ΕΝΑΛΛΑΚΤΙΚΕΣ ΔΙΑΔΡΟΜΕΣ")
93 print("=" * 78)
94 start = (3, 4, 5, 6)
95 seen, stack, edges, info = set(), [start], [], {}
96 while stack:
97     bs = stack.pop()
98     if bs in seen:
99         continue
100    seen.add(bs)
101    T, red, z = build(list(bs))
102    info[bs] = (point(list(bs), T), z)
103    for j in range(7):
104        if j in bs or red[j] <= 0:
105            continue
106        rat = [(T[i][7] / T[i][j], i) for i in range(m) if T[i][j] > 0]
107        if not rat:
108            continue
109        mn = min(r for r, _ in rat)
110        for r, i in rat:
111            if r != mn:
112                continue
113            nb = tuple(sorted([x for x in bs if x != bs[i] + [j]]))
114            edges.append((bs, nb, nm[j], nm[bs[i]], r == 0))
115            if nb not in seen:
116                stack.append(nb)
117
118 # ονομασία κορυφών
119 label, k = {}, 0
120 for bs in sorted(info, key=lambda t: float(info[t][1])):
121     p = info[bs][0]
122     if p not in label:
123         label[p] = "OABCDEFGHJIJ"[k]
124         k += 1
125 print("\nΚΟΡΥΦΕΣn κόμβοι( του γράφου):")
126 for p in sorted(label, key=lambda q: float(sum(c[i] * q[i] for i in range(3)))):
127     bl = [bs for bs in info if info[bs][0] == p]
128     z = sum(F(c[i]) * p[i] for i in range(3))
129     tag = f" <== ΕΚΦΥΛΙΣΜΕΝΗ ({len(bl)} βάσεις)" if len(bl) > 1 else ""
130     print(f" {label[p]}: ({', '.join(str(v) for v in p)}) Z = {z} βάσεις: "
131           + ", ".join("{ " + ", ".join(nm[j] for j in bs) + "}" for bs in bl) + tag)
132

```

```

133 print("\AKMEΣn επιτρεπτά( pivots):")
134 for a, bb, ein, lout, degen in sorted(edges, key=lambda e: (float(info[e[0]][1]), e[2])):
135     pa, za = info[a]
136     pb, zb = info[bb]
137     tag = " ΕΚΦΥΛΙΣΜΕΝΟ[ pivot: ίδιο σημείο, ίδιο Z]" if pa == pb else ""
138     print(f" {label[pa]}{{{','.join(nm[j] for j in a)}}} --({ein} in / {lout} out)--> "
139           f"{label[pb]}{{{','.join(nm[j] for j in bb)}}}   Z: {za} -> {zb}{tag}")
140
141 print(f"\Συνολικάn: {len(label)} κορυφές, {len(info)} βάσεις, {len(edges)} επιτρεπτά pivots.")

```

Α.7 ex_7.py — Φάση I με tableaux και πιστοποιητικό Farkas

```

1  """Εργασία
2  1 - Άσκηση 7Τεχνική
3  Φάσης I του αλγορίθμου Simplex δύο φάσεων.α
4
5  ()  -5x1 +  x2 - 3x3 + 3x4 + 8x5 <= -3
6      3x1 -  x2 + 2x3 -  x4 - 5x5 <=  2
7      -2x1 +  x2 -  x3          + 3x5 <=  2 ,  x >= 0β
8
9  ()  x1 +  x2 +  x3 +  x4 +  x5 =  2
10     -x1 + 2x2 +  x3 - 3x4 +  x5 =  1
11     x1 - 3x2 - 2x3 + 2x4 - 2x5 = -4 ,  x >= 0Στόχος
12
13  Φάσης I: min W' = Σ A_i ισοδύναμα( max W = Σ- A_i).Το
14  σύστημα είναι εφικτό <=> W* = 0 όλες( οι τεχνητές μηδέν).0
15  κώδικας τυπώνει όλα τα tableaux με ακριβή ρητή αριθμητική και, σε περίπτωση
16  μη εφικτότητας, εξάγει και το πιστοποιητικό Farkas από τη γραμμή
17  W γραμμικός( συνδυασμός των περιορισμών που δίνει αντίφαση).
18  """
19  from fractions import Fraction as F
20  import numpy as np
21  from scipy.optimize import linprog
22
23
24  def phasel(rows, rhs, senses, title):
25      """rows/rhs/senses: το αρχικό σύστημα. Επιστρέφει εφικτό(, BFS)."""
26      print("\n" + "=" * 78)
27      print(title)
28      print("=" * 78)
29      n = len(rows[0])
30      # 1) κάνουμε όλα τα b >= 0
31      R, B, S = [], [], []
32      for r, bb, s in zip(rows, rhs, senses):
33          if bb < 0:
34              R.append([-F(v) for v in r]); B.append(-F(bb))
35              S.append({"<=": ">=", ">=": "<=", "=": "="}[s])
36          else:
37              R.append([F(v) for v in r]); B.append(F(bb)); S.append(s)
38          print(f" {'+'.join(f'{v}x{j+1}' for j,v in enumerate(R[-1]))} {S[-1]} {B[-1]}")
39      # 2) slack / surplus / artificial
40      cols, names, basis = [], [f"x{j+1}" for j in range(n)], []
41      extra = []
42      for i, s in enumerate(S):
43          if s == "<=": # slack -> μπαίνει στη βάση
44              extra.append(("s", i, F(1)));
45          elif s == ">=": # surplus (-1) + τεχνητή
46              extra.append(("e", i, F(-1)))
47      # χτίσιμο πίνακα
48      T = [R[i][:] for i in range(len(R))]
49      for kind, i, sgn in extra:
50          names.append(f"{'s' if kind=='s' else 'e'}{i+1}")

```

```

51     for k in range(len(T)):
52         T[k].append(sgn if k == i else F(0))
53 art = []
54 for i, s in enumerate(S):
55     if s in ">=", "=":
56         names.append(f"A{i+1}"); art.append(len(names) - 1)
57         for k in range(len(T)):
58             T[k].append(F(1) if k == i else F(0))
59 # αρχική βάση
60 for i, s in enumerate(S):
61     if s == "<=":
62         basis.append(names.index(f"s{i+1}"))
63     else:
64         basis.append(names.index(f"A{i+1}"))
65 m, N = len(T), len(names)
66 tab = [T[i] + [B[i]] for i in range(m)]
67
68 def wrow():
69     # W = Σ- A_i -> γραμμή (c_j - z_j) με c_j = -1 για τεχνητές, 0 αλλού
70     cw = [F(-1) if j in art else F(0) for j in range(N)]
71     red = [cw[j] - sum(cw[basis[i]] * tab[i][j] for i in range(m)) for j in range(N)]
72     w = sum(cw[basis[i]] * tab[i][N] for i in range(m))
73     return red, w
74
75 it = 0
76 while True:
77     red, w = wrow()
78     print(f"\n--- Φάση I, επανάληψη {it}   Βάση = ({', '.join(names[j] for j in basis)}), W = {w}")
79     print(" " + "".join(f"{v:>7}" for v in names) + f"{'RHS':>7}")
80     for i in range(m):
81         print(f" {names[basis[i]]:<6}" + "".join(f"{str(tab[i][j]):>7}" for j in range(N))
82             + f"{str(tab[i][N]):>7}")
83     print(" c-z " + "".join(f"{str(red[j]):>7}" for j in range(N)) + f"{str(w):>7}")
84     cand = [j for j in range(N) if j not in basis and red[j] > 0]
85     if not cand:
86         break
87     e = max(cand, key=lambda j: red[j])
88     rat = [(tab[i][N] / tab[i][e], i) for i in range(m) if tab[i][e] > 0]
89     if not rat:
90         print(" μη φραγμένο δεν συμβαίνει στη Φάση I"); return None
91     mn = min(r for r, _ in rat)
92     i0 = [i for r, i in rat if r == mn][0]
93     print(f" εισερχόμενη {names[e]} (c-z={red[e]}); λόγοι: "
94         + ", ".join(f"{names[basis[i]]}: {tab[i][N]}/{tab[i][e]} = {tab[i][N]/tab[i][e]}"
95             for _, i in rat)
96         + f" -> εξερχόμενη {names[basis[i0]]}")
97     pv = tab[i0][e]
98     tab[i0] = [v / pv for v in tab[i0]]
99     for k in range(m):
100         if k != i0 and tab[k][e] != 0:
101             f = tab[k][e]
102             tab[k] = [tab[k][t] - f * tab[i0][t] for t in range(N + 1)]
103     basis[i0] = e
104     it += 1
105
106 red, w = wrow()
107 if w == 0:
108     x = [F(0)] * N
109     for i, bi in enumerate(basis):
110         x[bi] = tab[i][N]
111     print(f"\n W* = 0 => ΤΟ ΣΥΣΤΗΜΑ ΕΙΝΑΙ ΕΦΙΚΤΟ.")
112     print(" Αρχική ΒΕΦΛ για τη Φάση II: "
113         + ", ".join(f"{names[j]}={x[j]}" for j in range(N) if x[j] != 0)
114         + " όλες( οι υπόλοιπες = 0)")

```

```

115     print(f"  δηλαδή x = ({', '.join(str(x[j]) for j in range(n))})")
116     return True
117     print(f"\n W* = {w} < 0  =>  ΤΟ ΣΥΣΤΗΜΑ ΕΙΝΑΙ ΜΗ ΕΦΙΚΤΟ.")
118     inb = [names[j] for j in basis if j in art]
119     print(f"  τεχνητές που παραμένουν στη βάση με θετική τιμή: {inb}")
120     print("  η( γραμμή c-z των τεχνητών δίνει τους πολλαπλασιαστές του "
121           "πιστοποιητικού μηεφικτότητας- κατά Farkas)")
122     return False
123
124
125 # ----- α()
126 phase1([[-5, 1, -3, 3, 8], [3, -1, 2, -1, -5], [-2, 1, -1, 0, 3]],
127         [-3, 2, 2], ["<=", "<=", "<="],
128         "ΣΥΣΤΗΜΑ α()")
129 r = linprog([0]*5, A_ub=[[-5, 1, -3, 3, 8], [3, -1, 2, -1, -5], [-2, 1, -1, 0, 3]],
130            b_ub=[-3, 2, 2], bounds=[(0, None)]*5)
131 print(f"  Επαλήθευση scipy: status={r.status} εφικτό({' if r.status==0 else μη' εφικτό'})")
132
133 # ----- β()
134 phase1([[1, 1, 1, 1, 1], [-1, 2, 1, -3, 1], [1, -3, -2, 2, -2]],
135         [2, 1, -4], ["=", "=", "="],
136         "ΣΥΣΤΗΜΑ β()")
137 r = linprog([0]*5, A_eq=[[1, 1, 1, 1, 1], [-1, 2, 1, -3, 1], [1, -3, -2, 2, -2]],
138            b_eq=[2, 1, -4], bounds=[(0, None)]*5)
139 print(f"  Επαλήθευση scipy: status={r.status} εφικτό({' if r.status==0 else μη' εφικτό'})")
140
141 # Αλγεβρικό πιστοποιητικό μηεφικτότητας- για το β() κατά( Farkas).
142 # Οι συντελεστές των τεχνητών στην τελική γραμμή του A3 δίνουν τους
143 # πολλαπλασιαστές· εδώ προκύπτει ο απλούστατος συνδυασμός E1 + E2 + E3.
144 E = np.array([[1, 1, 1, 1, 1], [-1, 2, 1, -3, 1], [1, -3, -2, 2, -2]])
145 rhs = np.array([2, 1, -4])
146 mult = np.array([1, 1, 1])
147 print("\n ΠΙΣΤΟΠΟΙΗΤΙΚΟ ΜΗΕΦΙΚΤΟΤΗΤΑΣ- (Farkas) για το σύστημα β():")
148 print(f"  E1 + E2 + E3 -> συντελεστές {mult @ E} = {mult @ rhs}")
149 print("  δηλαδή x1 = -1 < 0,  που αντιβαίνει στον περιορισμό x1 >= 0.")
150 print("  Άρα κανένα x >= 0 δεν ικανοποιεί ταυτόχρονα τις τρεις εξισώσεις.")

```